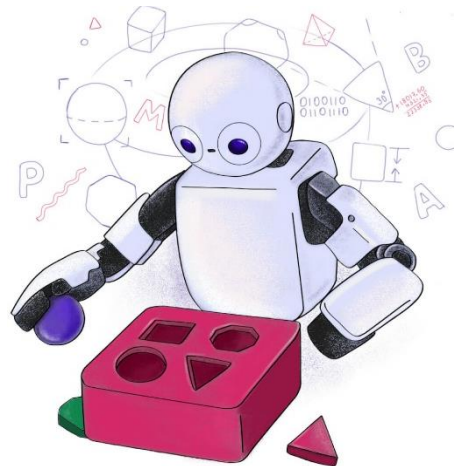


C24 - Inteligência Artificial: *k-Means*



Recapitulando

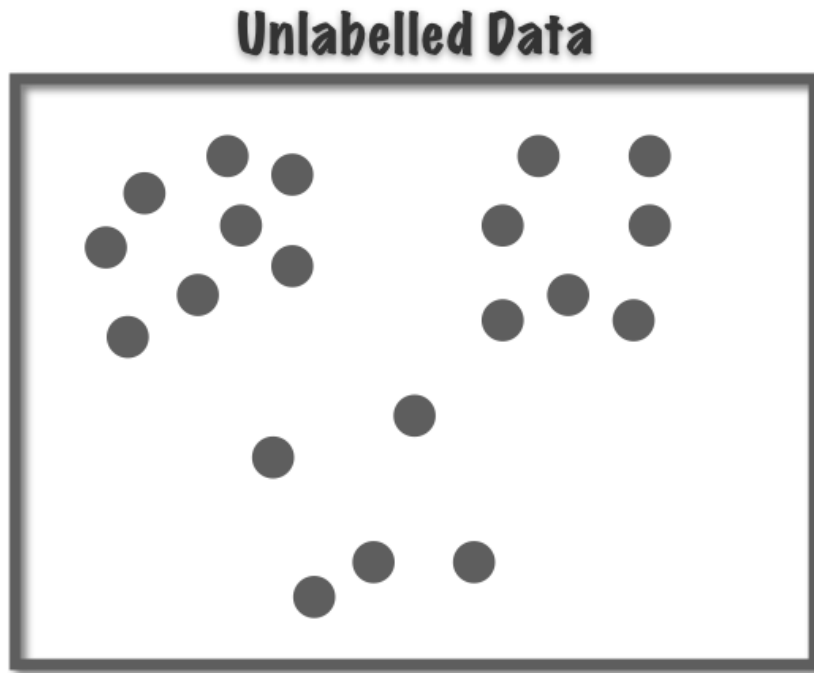
- Até o momento, todos os algoritmos que aprendemos seguiam o paradigma do aprendizado supervisionado.
- Hoje, falaremos sobre ***clustering***, que são algoritmos de **aprendizado não-supervisionado**.
 - Não temos rótulos, apenas atributos.
- Eles visam criar agrupamentos de dados (chamados de ***clusters*** ou **grupos**) segundo seu **grau de semelhança**.
- Nesta aula, aprenderemos sobre o algoritmo chamado de **k-Médias** (ou ***k-Means***, em inglês) que é um dos algoritmos mais simples de ***clustering***.

clustering be like:



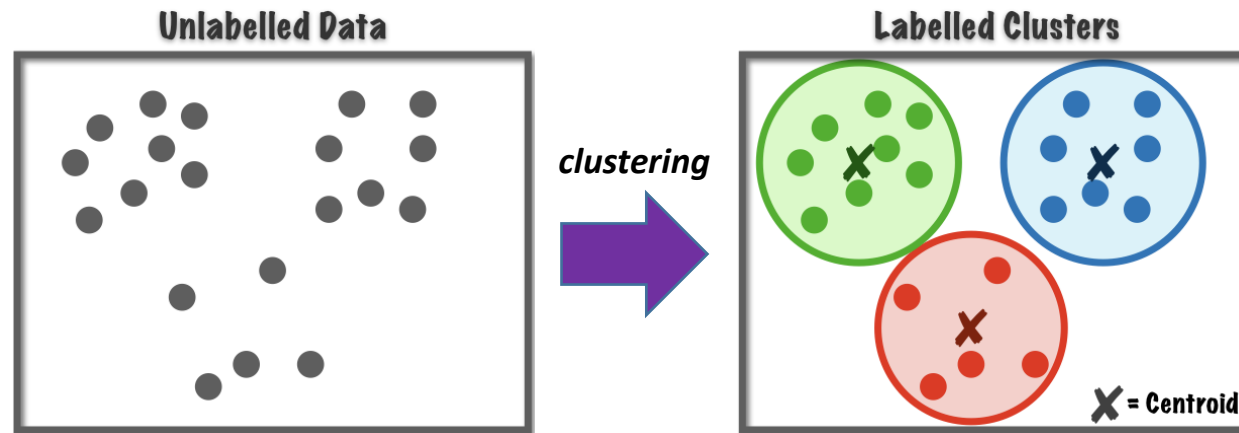
A **clusterização** organiza dados como na figura: cada ponto encontra seu grupo mais próximo, formando *clusters*.

Motivação



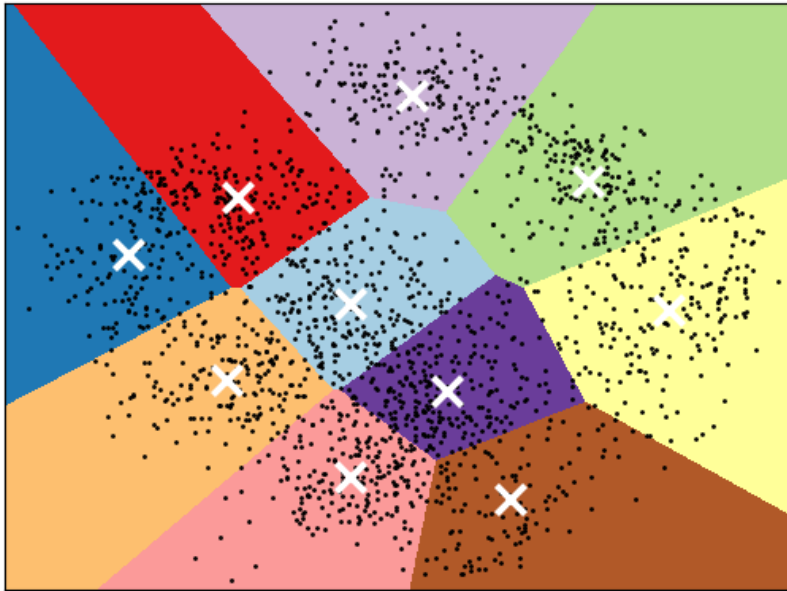
- O que podemos fazer se não tivermos informações sobre as classes (i.e., rótulos) a que pertencem os exemplos de entrada?

Motivação



- Veremos que informações úteis podem ser obtidas mesmo de exemplos cujas classes não são conhecidas.
- Enquanto o **aprendizado supervisionado** se concentra na **indução de classificadores**, o **aprendizado não-supervisionado** está interessado em **extrair estrutura e conhecimento de dados quando não existem rótulos**.
- *Clustering* tenta reproduzir como humanos organizam o mundo: agrupamos objetos por similaridade

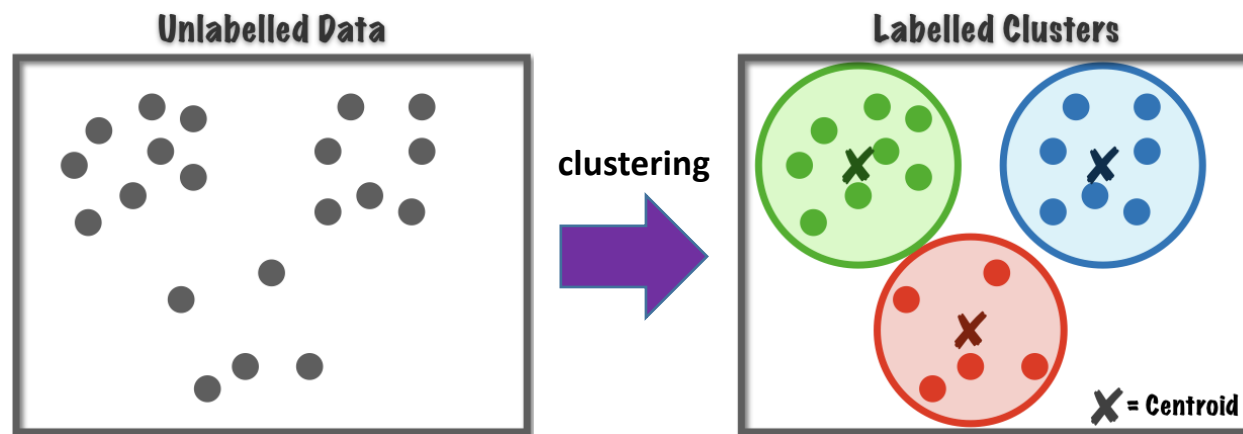
Motivação



- A tarefa mais popular dos algoritmos deste paradigma é a procura por **grupos** (chamados *clusters*) de **exemplos semelhantes**.
- O *clustering* agrupa dados **com base em similaridade** sem precisar de supervisão.
- A ideia mais importante é revelar a **estrutura intrínseca dos dados**:
 - padrões escondidos
 - subgrupos naturais
 - relações não explícitas

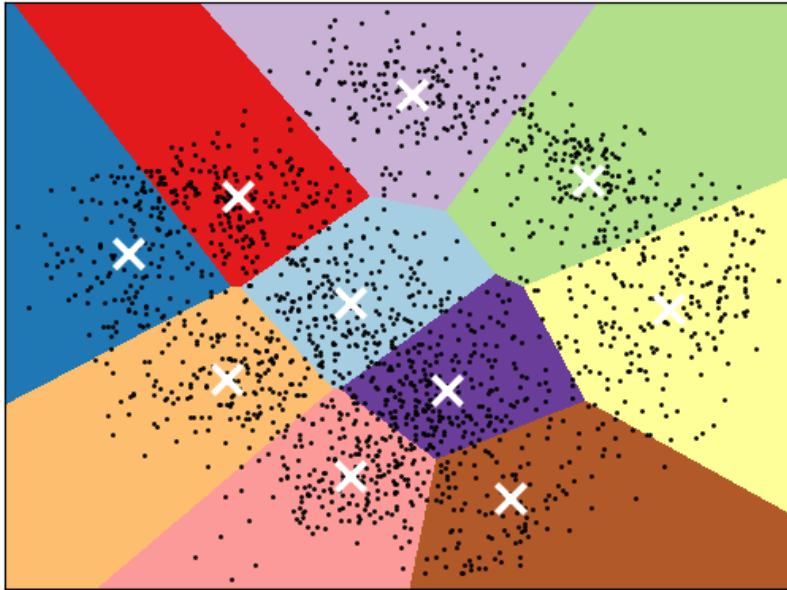
Identificação de *clusters*

- A **tarefa fundamental** do **aprendizado não-supervisionado** é a **identificação de *clusters***.
- Nessa tarefa, a **entrada** é um conjunto de **vetores de atributo** (i.e., exemplos), **mas sem rótulos**.
- A **saída** é um conjunto de ***clusters*** a que pertencem cada um dos exemplos do conjunto de treinamento.



OBS.: A identificação visual de clusters em um espaço bidimensional é fácil, mas em quatro ou mais dimensões isso já não é mais possível. Nesses casos, apenas algoritmos de identificação de clusters conseguem agrupar os dados.

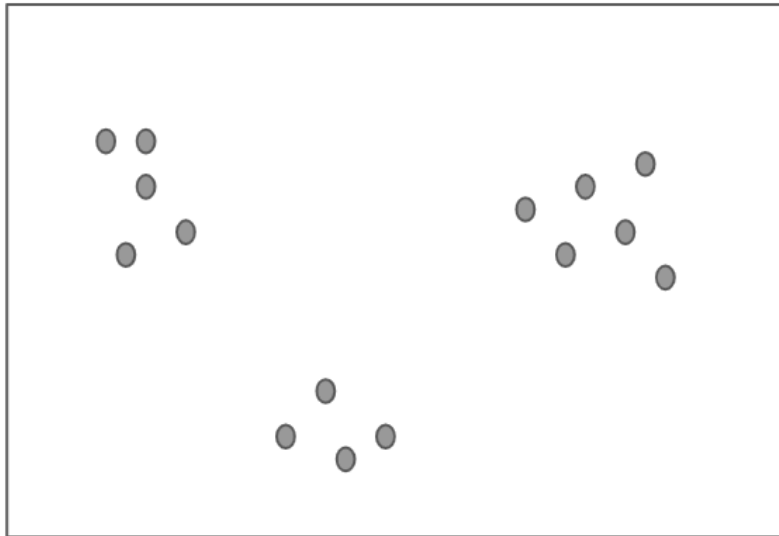
Aplicações



- Redução de complexidade
 - substituir milhões de pontos por alguns grupos
 - resumir dados complexos
- Pré-processamento para outros modelos
 - criar features (cluster ID)
 - inicializar modelos
 - reduzir dimensionalidade
 - organizar dados antes de aprendizado supervisionado
- Detecção de padrões raros
 - detector de comportamento “normal”
 - tudo fora disso é considerado suspeito

Como representar os *clusters*?

Unlabelled Data



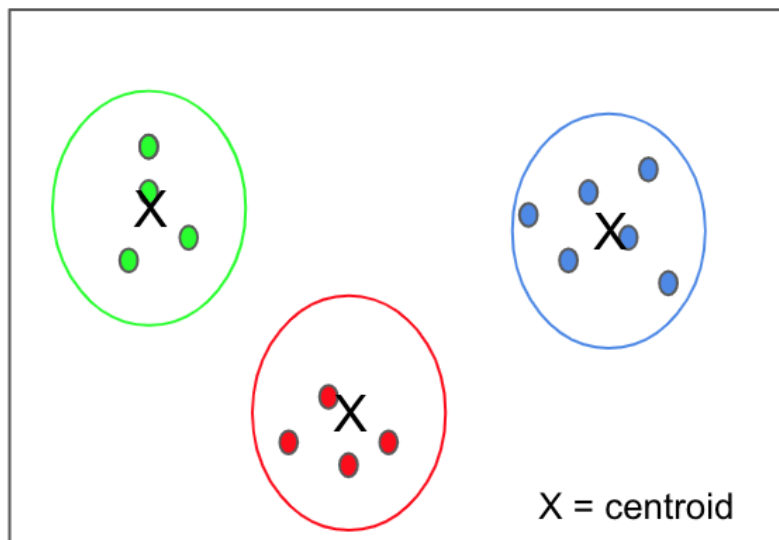
- Para realizar a identificação, temos que **decidir como os *clusters* serão representados**.
- Existem algumas opções como a densidade, distribuição, hierarquia, etc.
- Porém, a abordagem mais simples usa os **centroides** (i.e., centros) dos clusters.
- O algoritmo conhecido como ***k-Means*** implementa esta abordagem.
- Vejamos então suas características e como funciona.

Centroide

- **Centroide** é o ponto central de um *cluster* de dados.
 - Representa a "média" das amostras dentro do cluster.
- Os **centroides** podem ser usados como
 - centros para inicializar redes neurais Bayesianas ou de funções de base radial (RBF),
 - estimativas de valores de atributos ausentes.
 - forma de comprimir dados usando apenas o centróide mais próximo.
 - forma de visualizar dados multidimensionais.
 - *feature engineering*: o índice do centróide (ID) e a distância até os centróides podem ser novos atributos.
 - forma de detectar anomalias: distância alta ao centróide indica *outlier*.
 - forma de agrupar dados sem classes em problemas semi-supervisionados: os *clusters* são usados como classes e, conseqüentemente, rotular os dados.

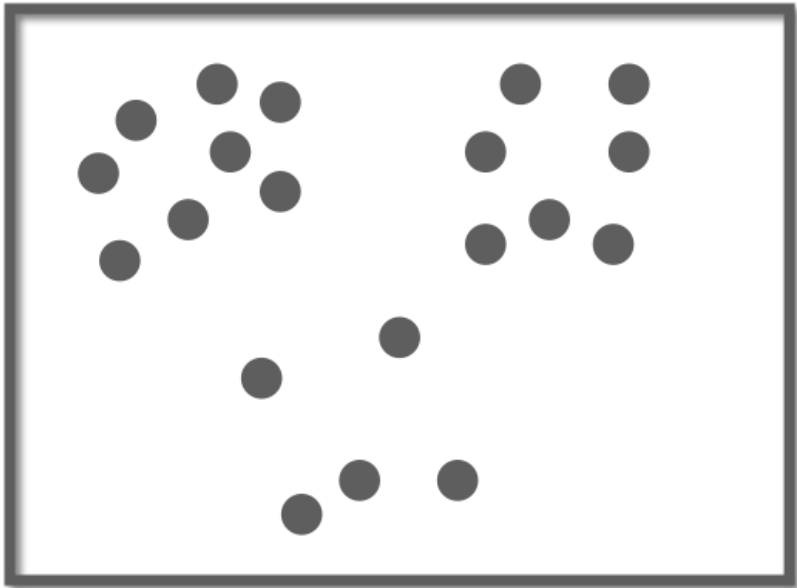
Como representar os *clusters*?

Labelled Data



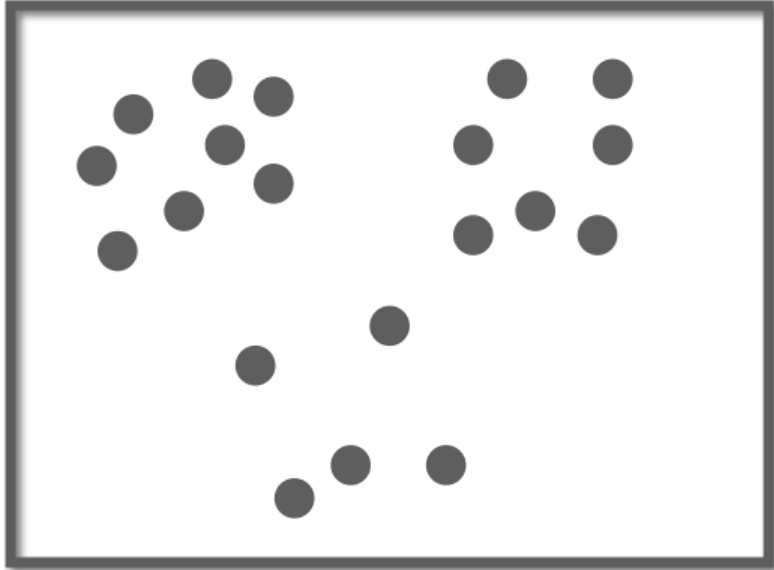
- Por exemplo, suponhamos os seguintes **vetores de atributos** em um espaço bidimensional, x_1 e x_2 : (2, 5), (1, 4), (3, 6).
- Nesse caso, se elas pertencem a um *cluster*, seu **centroide** é representado pelo vetor (2, 5), pois
 - A média do primeiro atributo é $\frac{2+1+3}{3} = 2$.
 - A média do segundo atributo é $\frac{5+4+6}{3} = 5$.
- **OBS.:** Em geral, aplicamos clusterização apenas a atributos numéricos.
 - Existem algoritmos de clusterização específicos para dados categóricos: *k-modes* e *k-prototypes*.

Quantos devem ser os *clusters*?



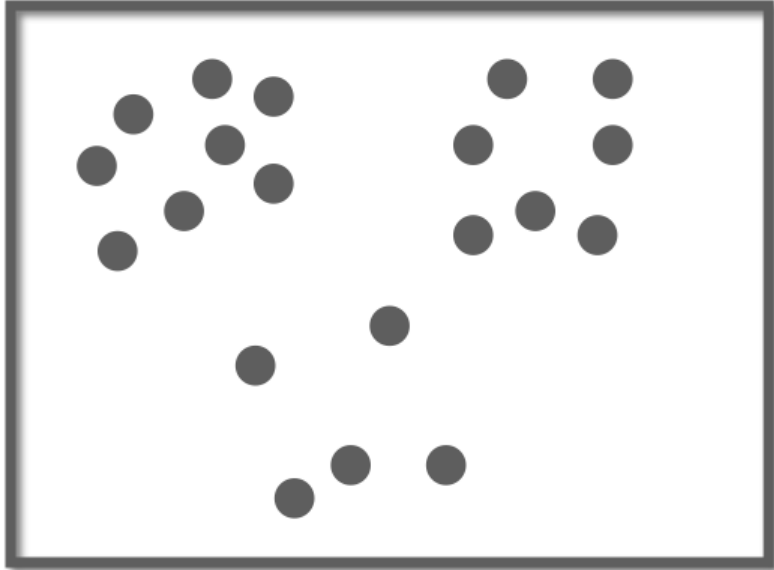
- Os *clusters* não devem se sobrepor.
- Cada amostra deve **pertencer** a **um e apenas um *cluster***.
- Porém, **dentro do mesmo cluster**, as **amostras** devem estar relativamente **próximas** umas das outras e **distantes** das **amostras dos outros *clusters***.
- Aí surge uma dúvida.
- Quantos *clusters* um conjunto de dados contém?

Quantos devem ser os *clusters*?



- Na figura conseguimos identificar visualmente três clusters.
- No entanto, o número de opções existentes não se limita a essa única possibilidade:
 - em um extremo, todo o conjunto de dados pode ser pensado como formando um grande cluster;
 - no outro, cada exemplo pode ser visto como representando seu próprio cluster de um único exemplo.

Quantos devem ser os *clusters*?



- **Implementações** práticas geralmente evitam esse problema pedindo ao **usuário que forneça o número de *clusters***.
- Existem diversos métodos para otimizar o hiperparâmetro que define o número de *clusters*, dentre eles, o mais usado é o método do cotovelo.
- Veremos como ele funciona em breve.
- Porém, antes, precisamos entender como os *clusters* são criados.

Medindo distâncias

- Algoritmos para identificação de *clusters* precisam de uma **métrica para avaliar a distância entre uma amostra e o centroide** de um *cluster*.
- Uma forma de fazer isso é usar a **distância euclidiana** entre os dois vetores

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^K (x_i - y_i)^2},$$

onde K é o número de atributos e x_i e y_i são os i -ésimos elementos dos vetores tendo sua distância calculada.

A qual *cluster* um exemplo deve pertencer?

- Vamos supor que existam N clusters cujos centroides são denotados pelo vetor, $\mathbf{c}_i, \forall i \in (1, N)$.
- Uma amostra $\mathbf{x}$ tem uma certa distância $d(\mathbf{x}, \mathbf{c}_i)$ para cada centroide.
- Se $d(\mathbf{x}, \mathbf{c}_p)$ é a menor dessas distâncias, então, é natural colocarmos $\mathbf{x}$ como pertencente ao centroide $\mathbf{c}_p$, ou seja, o p -ésimo cluster.
- Portanto, escolhemos a **menor distância** para **definir a qual cluster um exemplo pertence**.
- Agora, veremos como funciona o algoritmo *k-Means*.

k-Means

- O "*k*" é um hiperparâmetro que define o número de *clusters*.
- O pseudocódigo do algoritmo é mostrado abaixo.

Entradas: conjunto de exemplos e número de *clusters*, *k*.

1. Defina *k* centroides iniciais (geralmente, feito de forma aleatória).

2. Repita

- a) Calcule a distância de cada exemplo, *x*, para cada um dos *k* centroides e atribua cada exemplo ao *cluster* mais próximo.
- b) Calcule o novo centroide de cada *cluster**

3. Enquanto as posições dos centroides continuarem mudando.

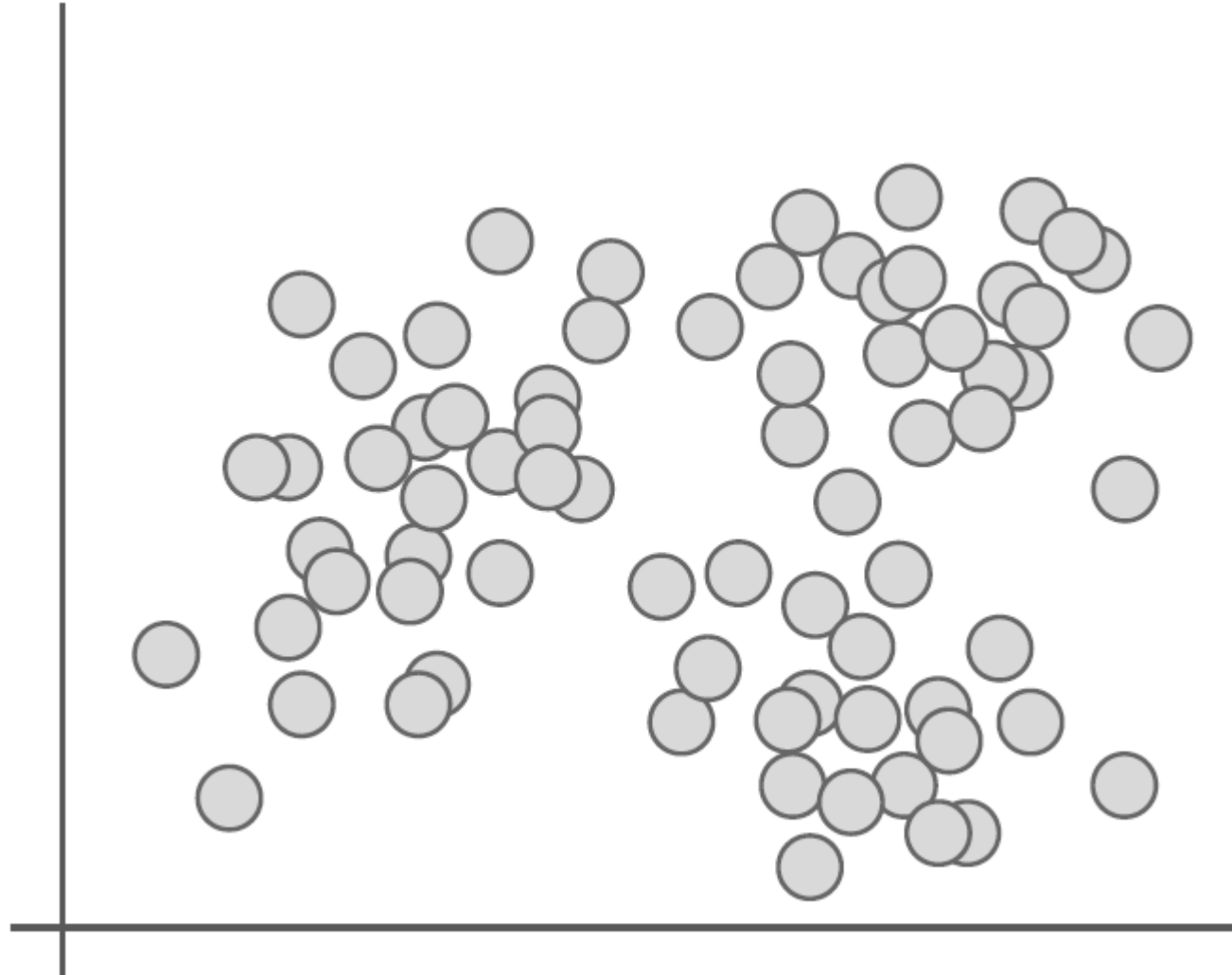
*os novos centroides são calculados como a média aritmética (centro geométrico) das amostras pertencentes a cada *cluster*.

- O algoritmo garantidamente chega a uma situação em que cada exemplo se encontra no *cluster* mais próximo, de modo que, a partir deste momento, os centroides não mudem mais.

Vejam os um exemplo

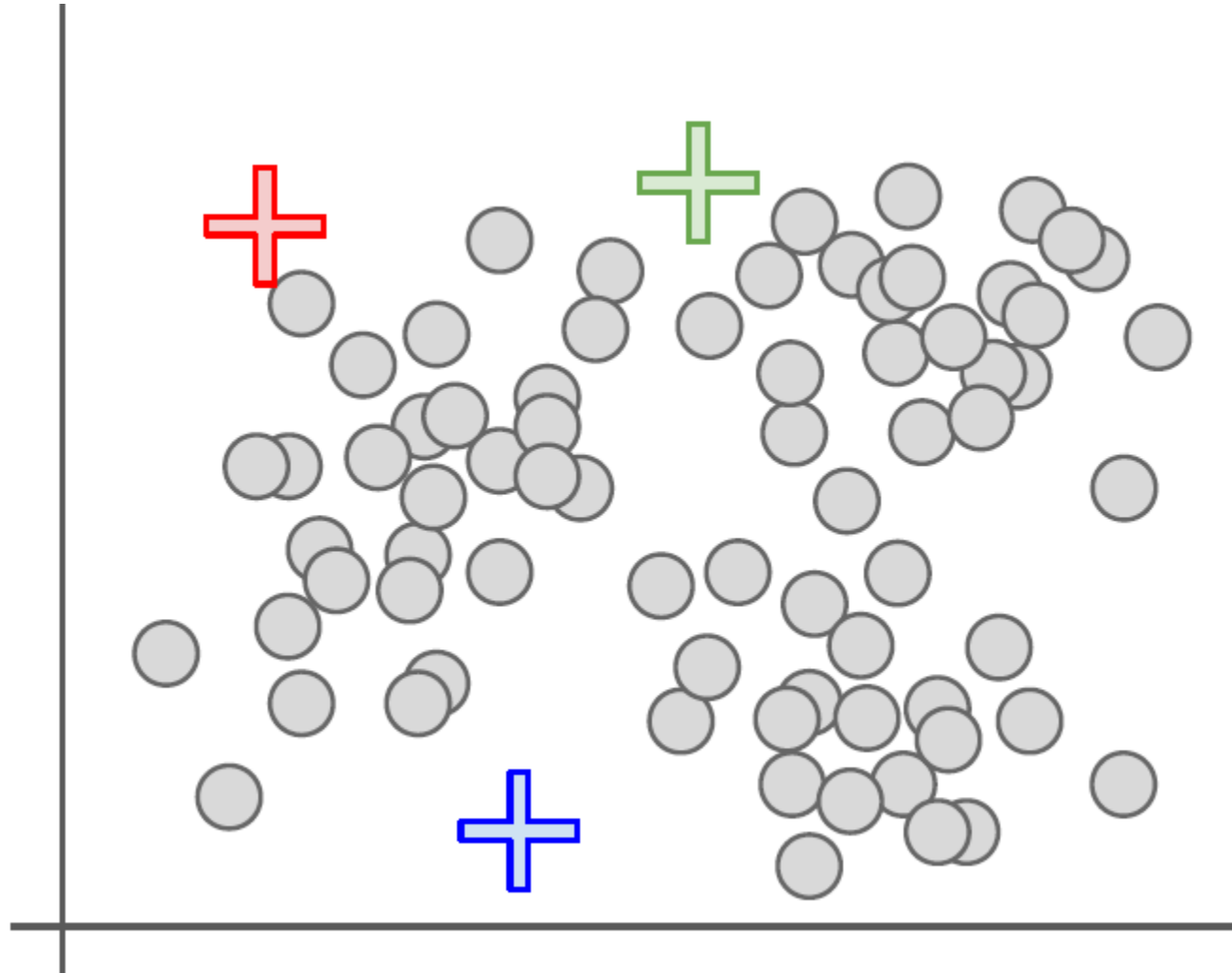
Funcionamento do *k-Means*

1. Define-se o número de clusters, $k = 3$



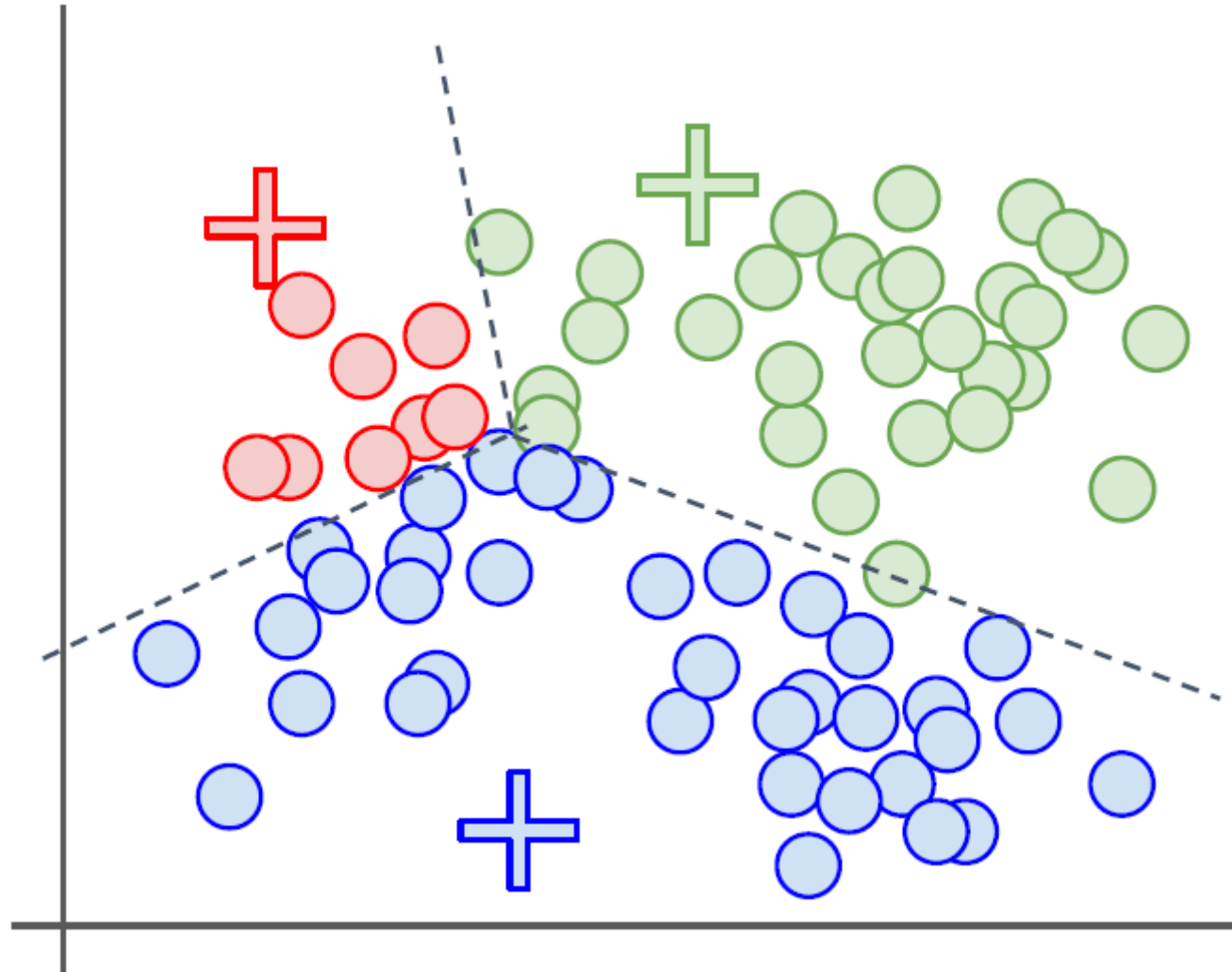
Funcionamento do *k-Means*

2. Escolhe-se os centroides para cada cluster de forma aleatória.



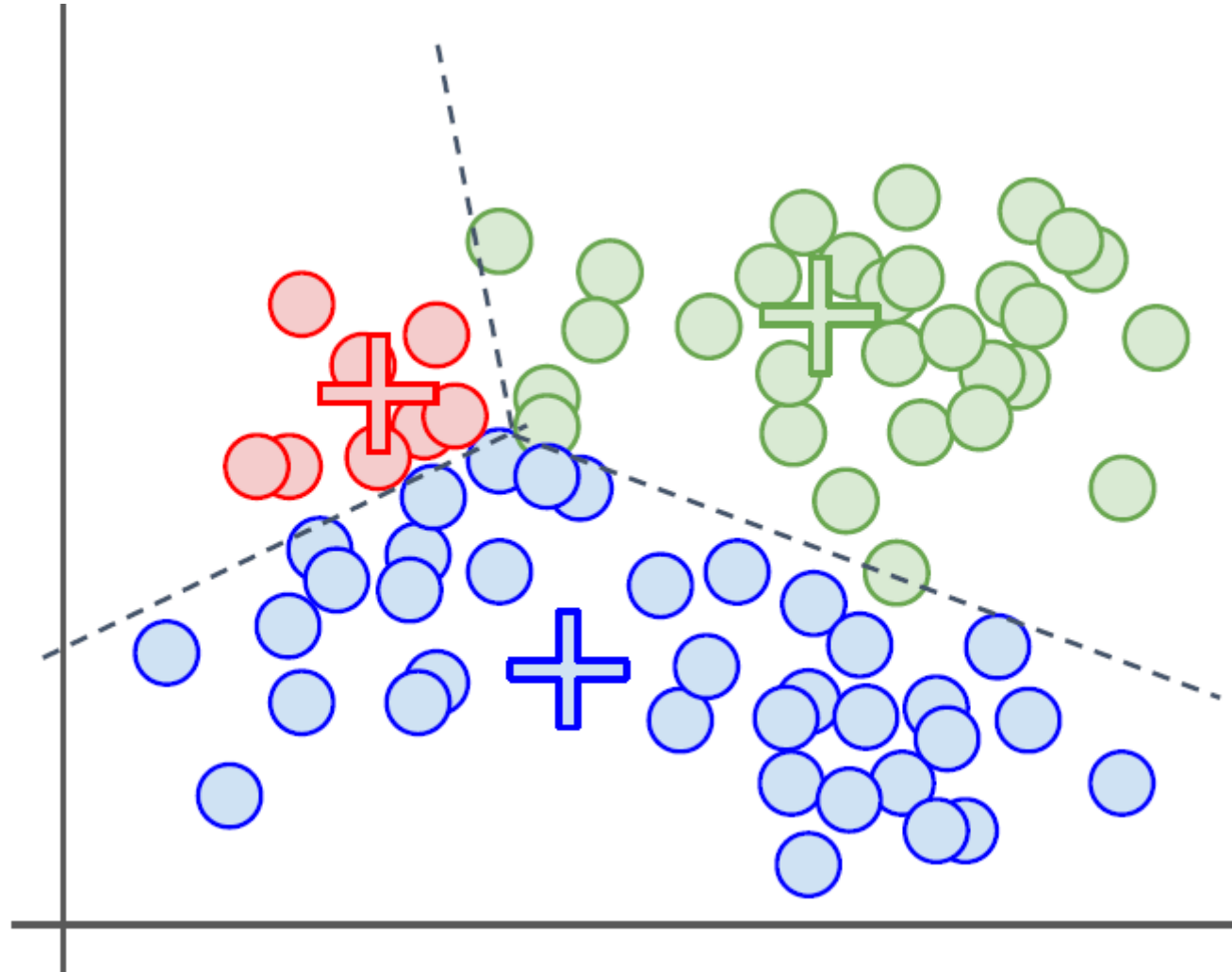
Funcionamento do *k-Means*

3. Atribui-se cada amostra ao centroide mais próximo com base na distância euclidiana.



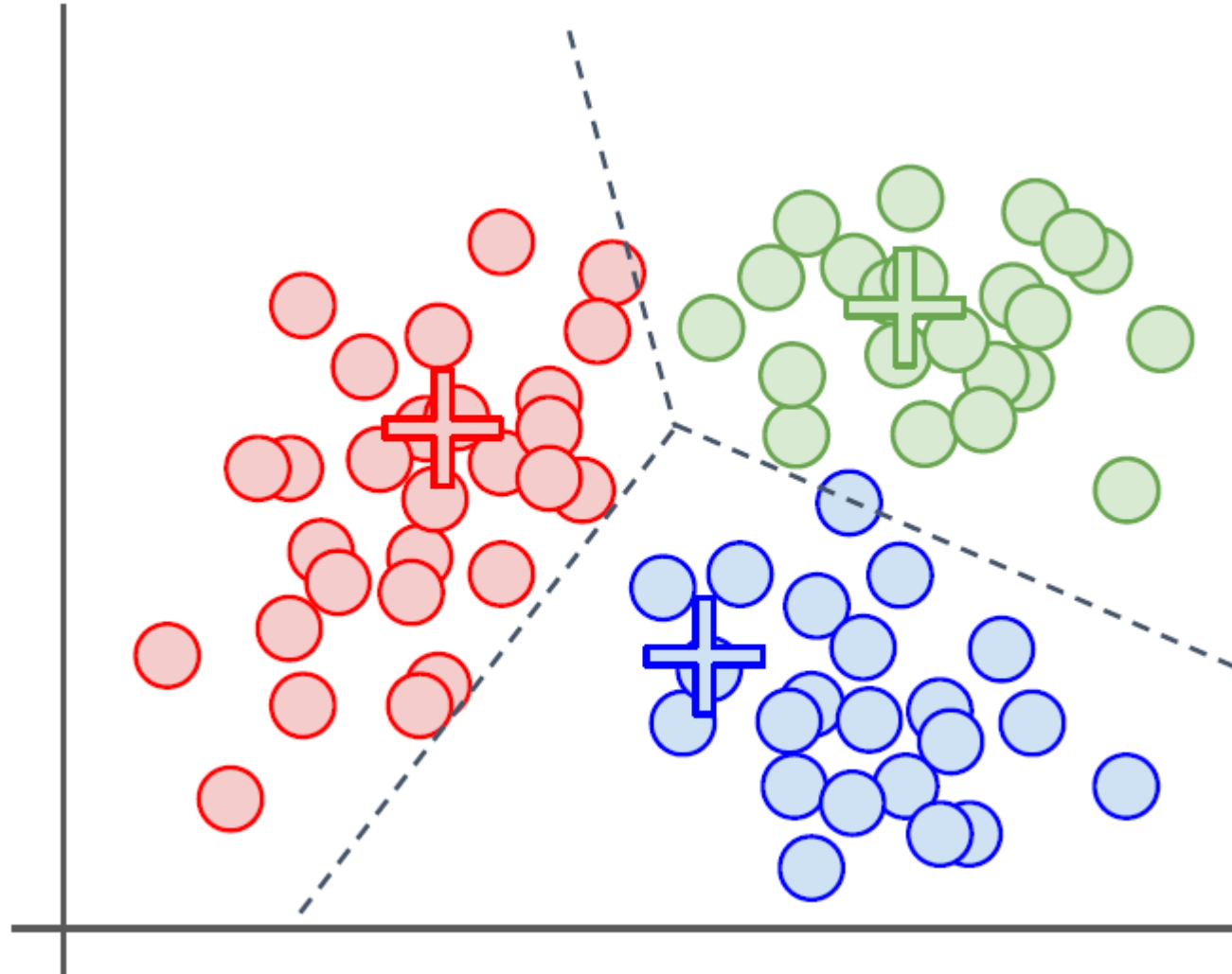
Funcionamento do *k-Means*

4. Recalcula-se o centroide dos clusters.



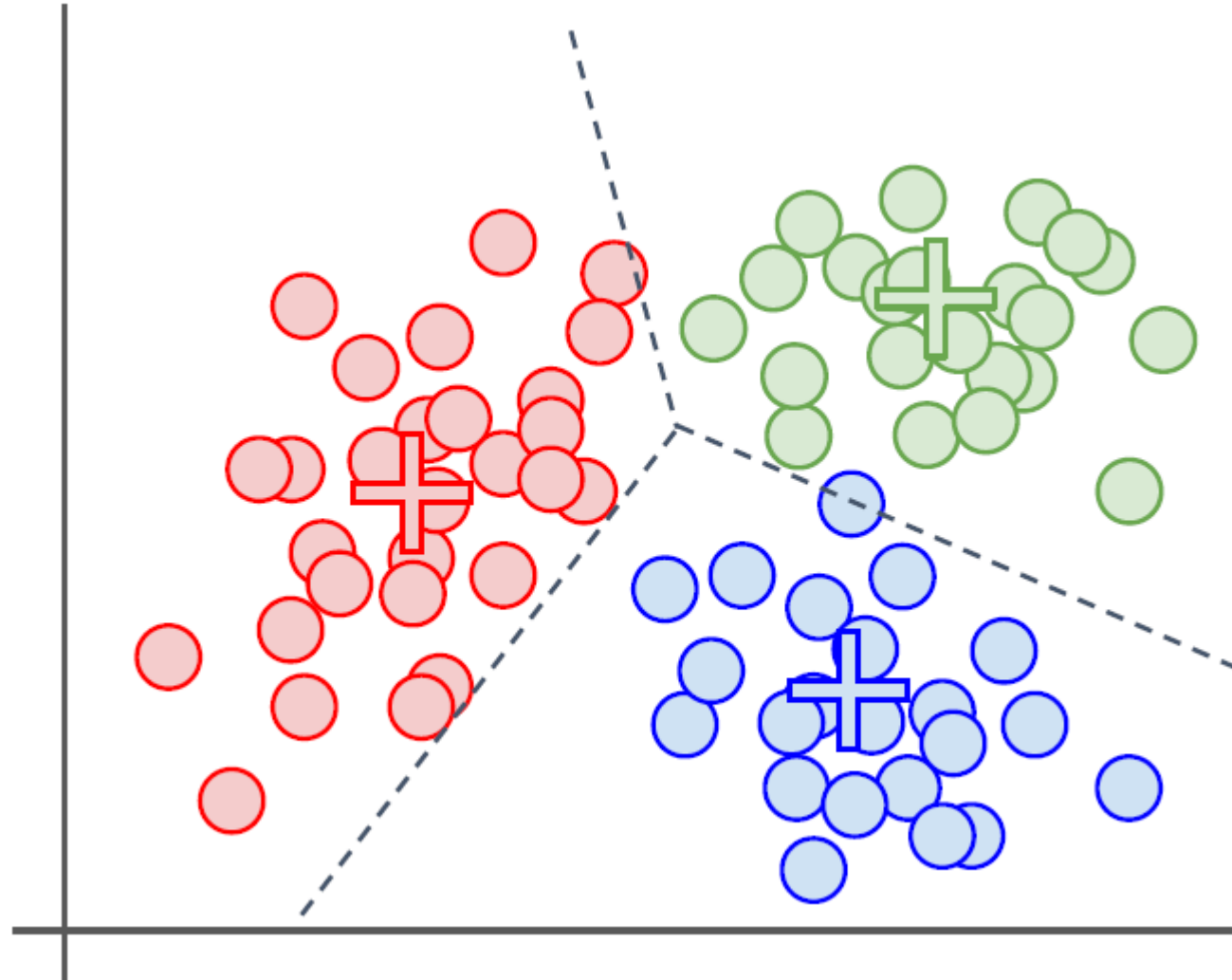
Funcionamento do *k-Means*

5. Repete-se os passos 3 e 4.



Funcionamento do *k-Means*

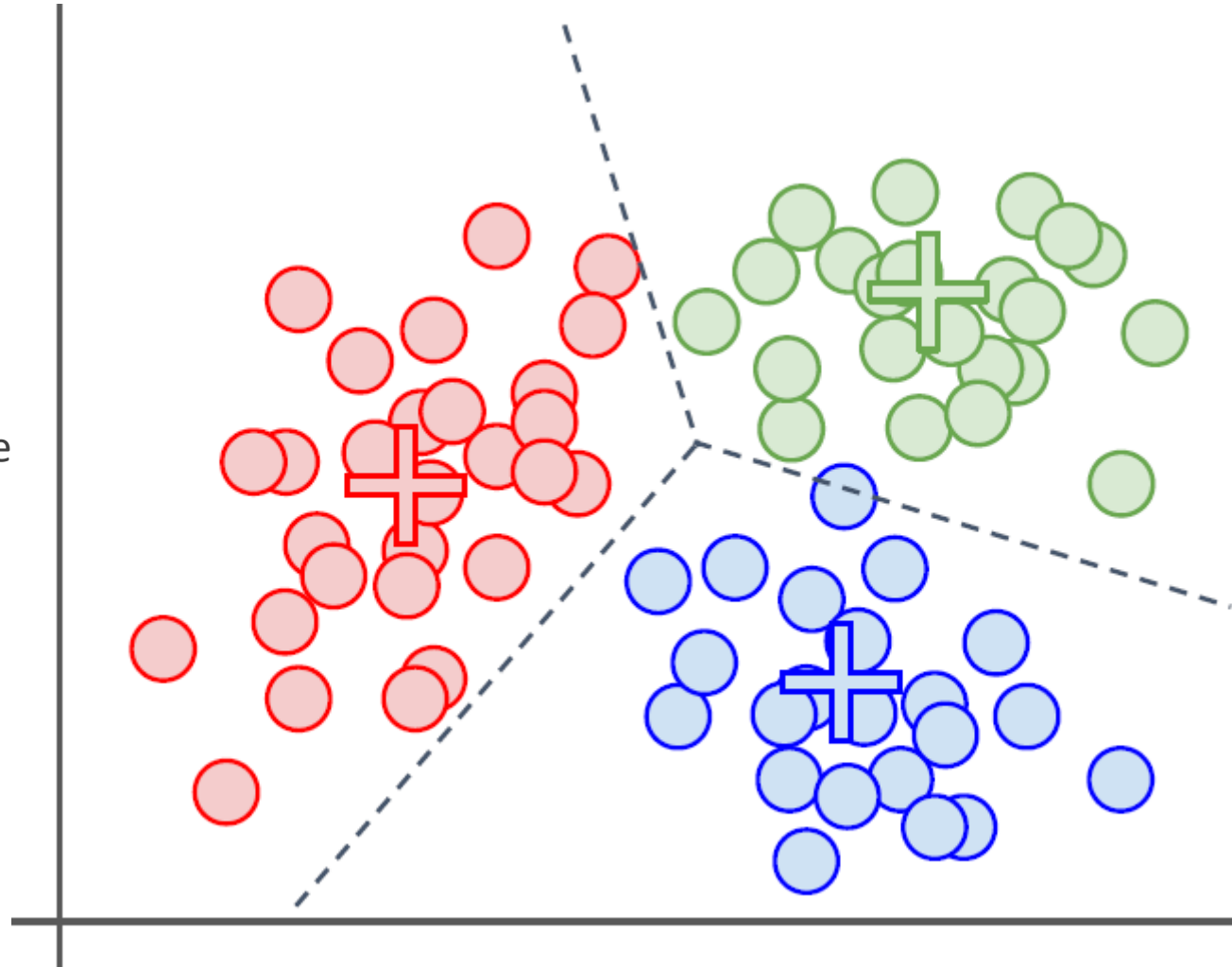
5. Repete-se os passos 3 e 4.



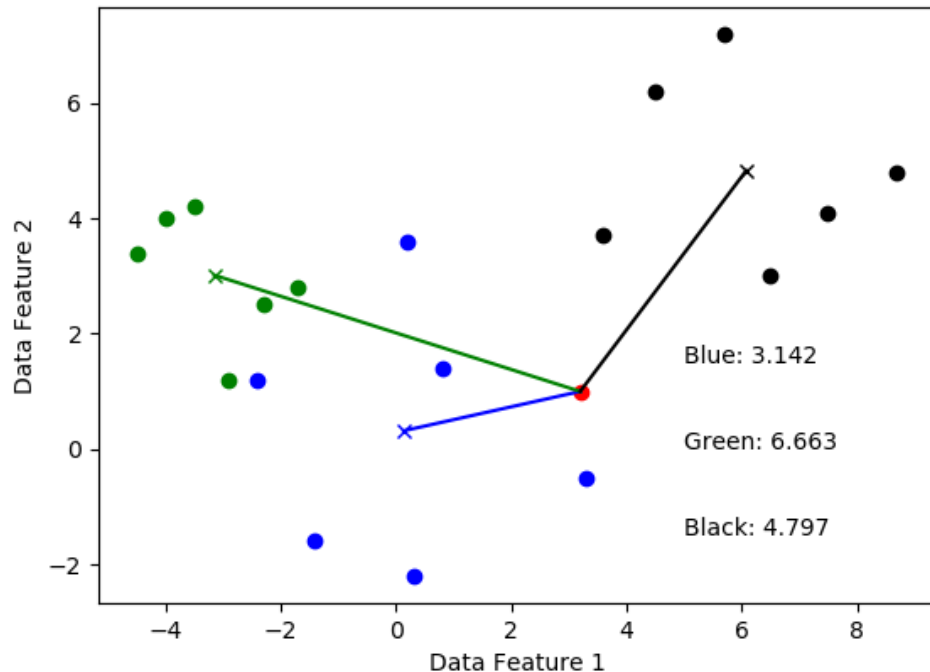
Funcionamento do *k-Means*

Repete-se os passos 3-4 até que um dos casos abaixo ocorra:

1. A soma das distâncias entre as amostras e o centroide correspondente seja minimizada
2. Não haja alteração nos centroides
3. O número máximo de iterações seja atingido

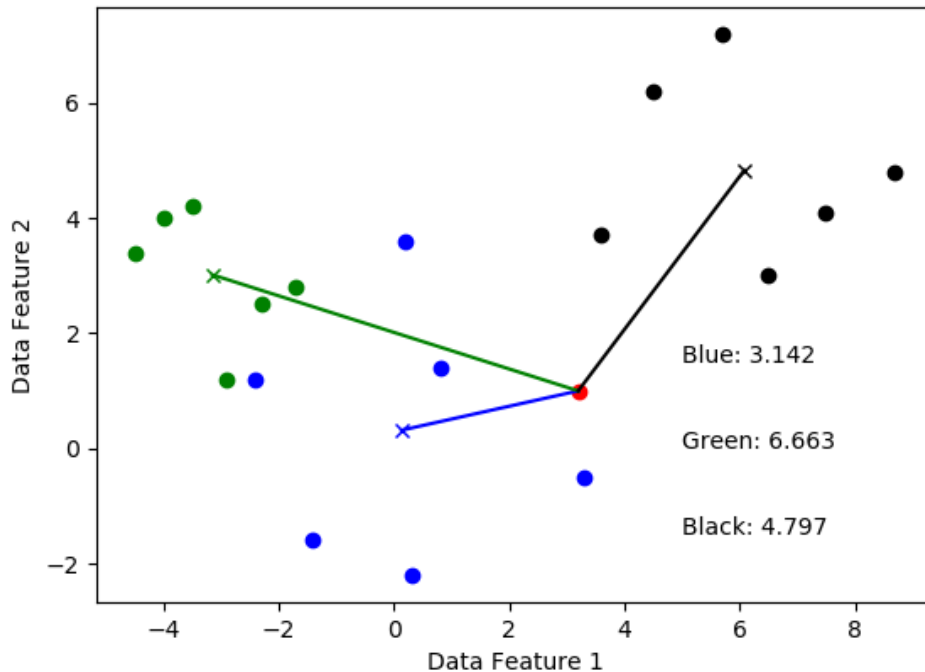


Como inicializar os centroides?



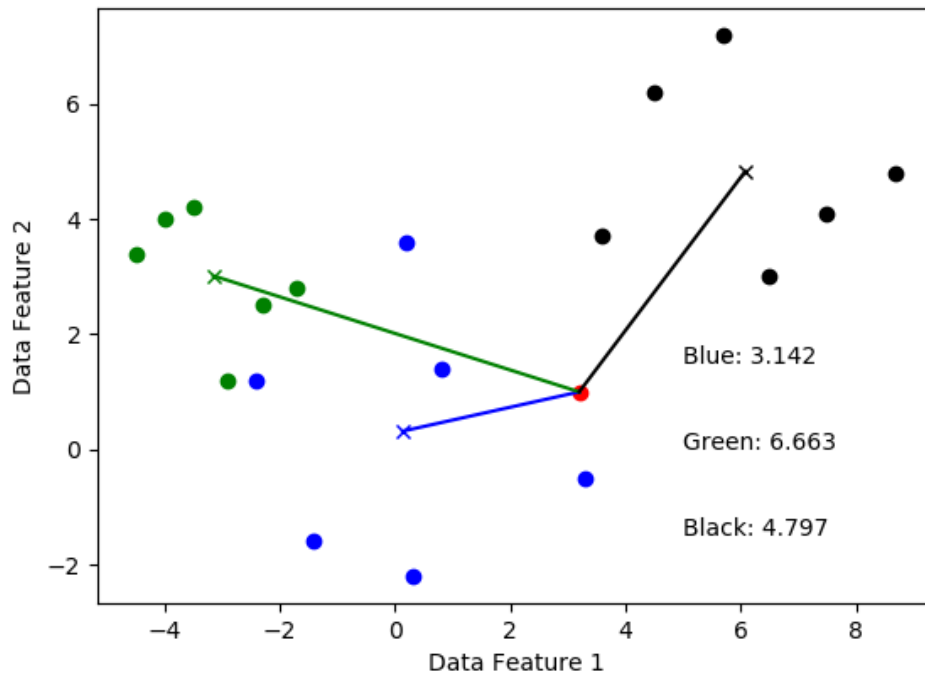
- O procedimento mais simples para inicializar os centroides é **escolher k exemplos de treinamento aleatórios** e os considerar como os centroides iniciais.
 - Pode escolher pontos muito próximos
 - Rodar várias vezes e escolher o melhor resultado
- Os **clusters iniciais** são então criados associando cada um dos exemplos ao seu **centroide mais próximo**.
- Cada centroide define uma região e as amostras são distribuídas com base nisso.

Como inicializar os centroides?



- Se os centroides iniciais mudam:
 - muda completamente a divisão inicial
 - muda quais amostras vão para qual *cluster* no início.
 - pode convergir para agrupamentos diferentes para cada execução.
- O número de **transferências** de um exemplo **de um *cluster* para outro** **depende dos centroides iniciais**.
 - Todo o processo depende do ponto de partida.
- Se os **centroides iniciais** já forem **perfeitos**, **nenhum exemplo precisa ser atribuído a outro *cluster*** e o algoritmo é encerrado.

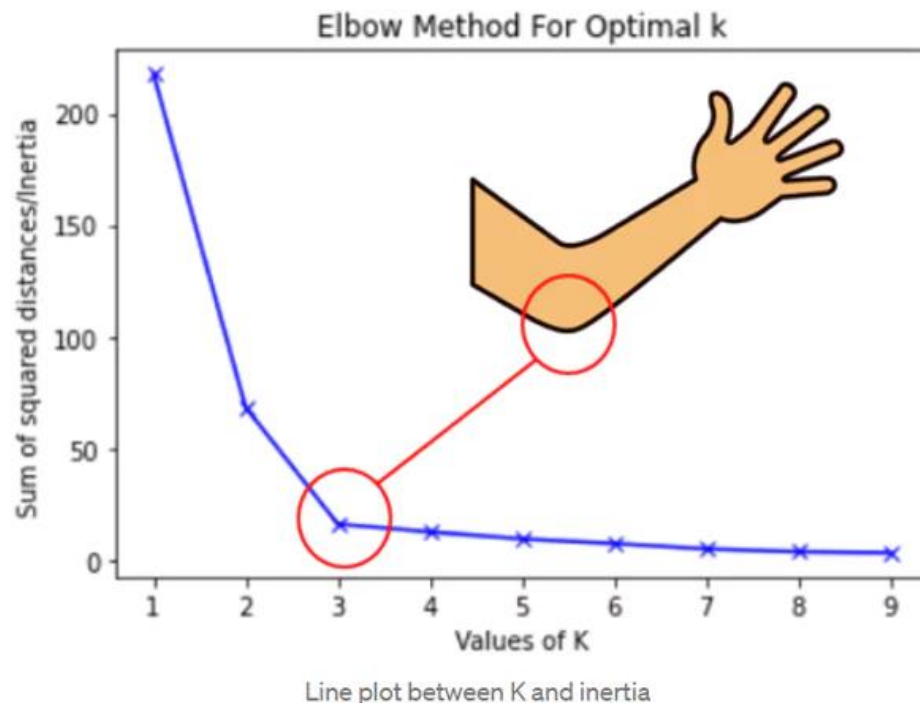
Como inicializar os centroides?



- Portanto, a **inicialização é importante** no sentido de que **um ponto de partida melhor** garante que a **solução seja encontrada mais rápido**.
- Ou seja, o **tempo de convergência** do algoritmo **depende da inicialização**.
- O algoritmo é **altamente sensível à escolha inicial dos centroides**.

Como definimos o número de
clusters?

Método do cotovelo



- O método busca o ponto onde a **melhoria do aumento de *clusters* começa a diminuir significativamente**.
- Ou seja, aumentar k sempre melhora o ajuste, mas chega um ponto em que o ganho é **marginal**.
 - No limite, quando k igual a N , a inércia é 0 (mínima possível), mas o modelo será extremamente complexo (overfitting).
- Esse ponto é o “**cotovelo**” da curva.
- Que métrica usamos para plotar a curva?

Soma dos quadrados intra-*clusters*

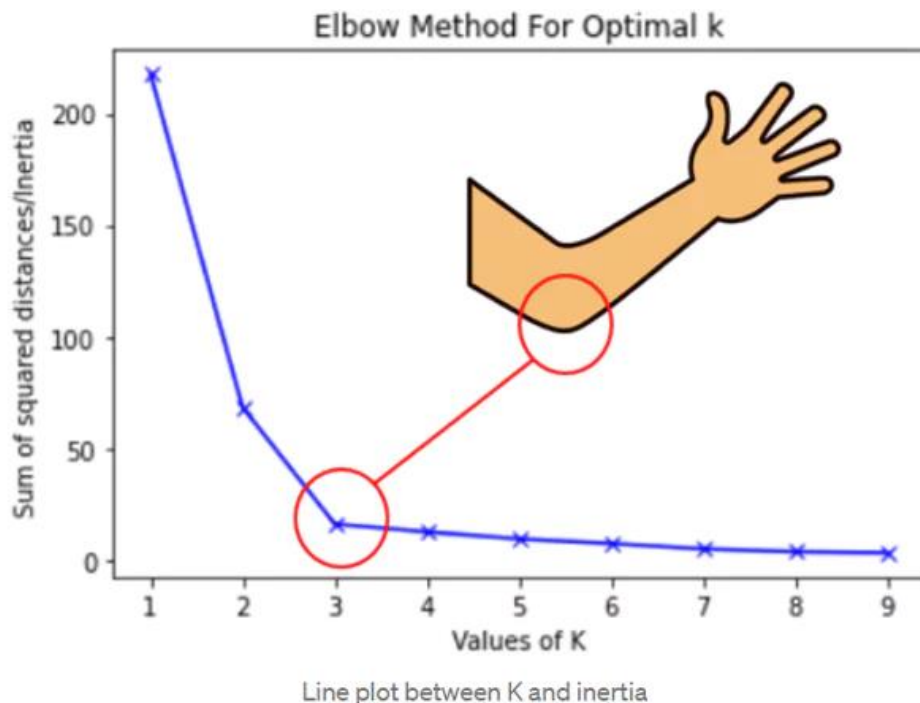
- Usa-se a soma dos quadrados intra-*clusters*, também conhecida como inércia

$$I(k) = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

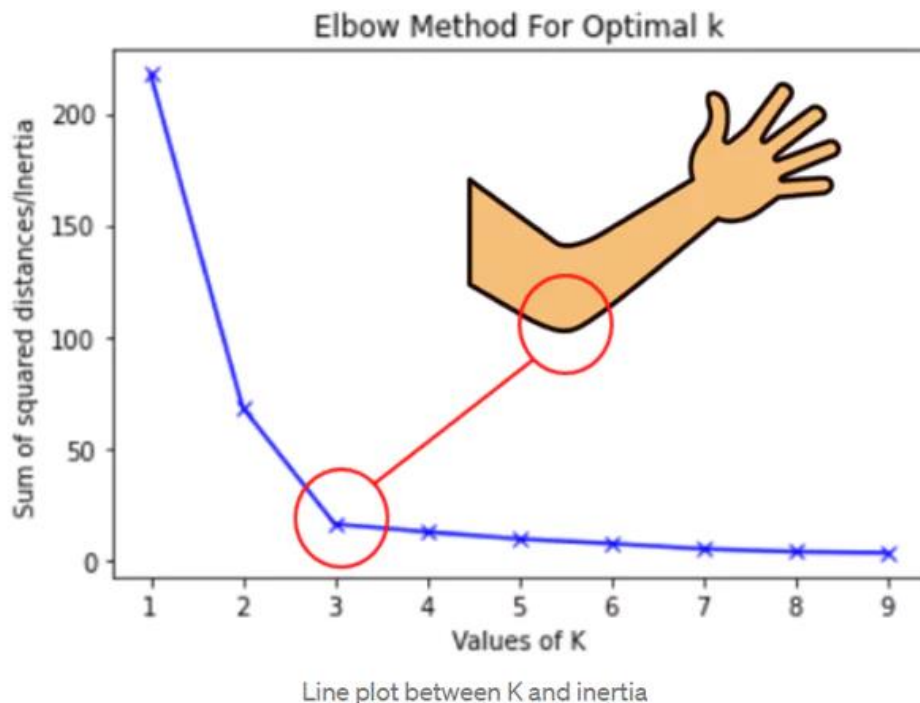
onde μ_i é o centroide do i -ésimo *cluster*, C_i é conjunto de amostras no i -ésimo *cluster* e $\|x - \mu_i\|^2$ é a distância até μ_i .

- Ela mede o quão “compactos” são os *clusters*.

- quanto menor → melhor

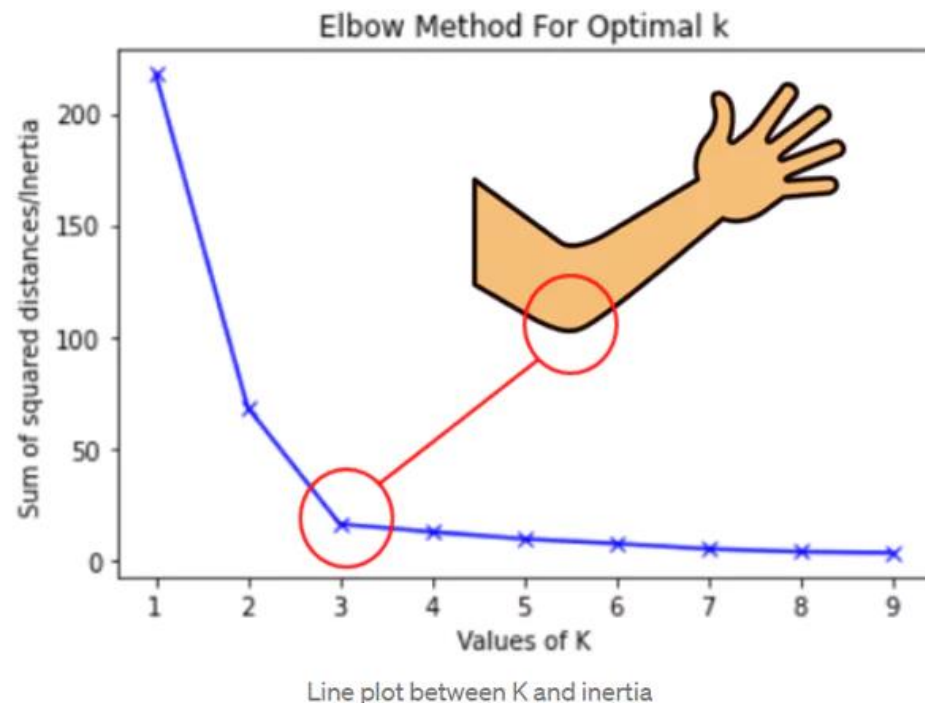


Soma dos quadrados intra-*clusters*



- Plota-se o valor da inércia para um intervalo de valores de k .
- A curva resultante quantifica o quão próximos os pontos de um cluster estão do seu centroide.
- Quanto menor a inércia, mais coesos são os *clusters*:
 - os pontos dentro de um *cluster* estão fortemente agrupados, próximos entre si e do centroide.

Soma dos quadrados intra-*clusters*

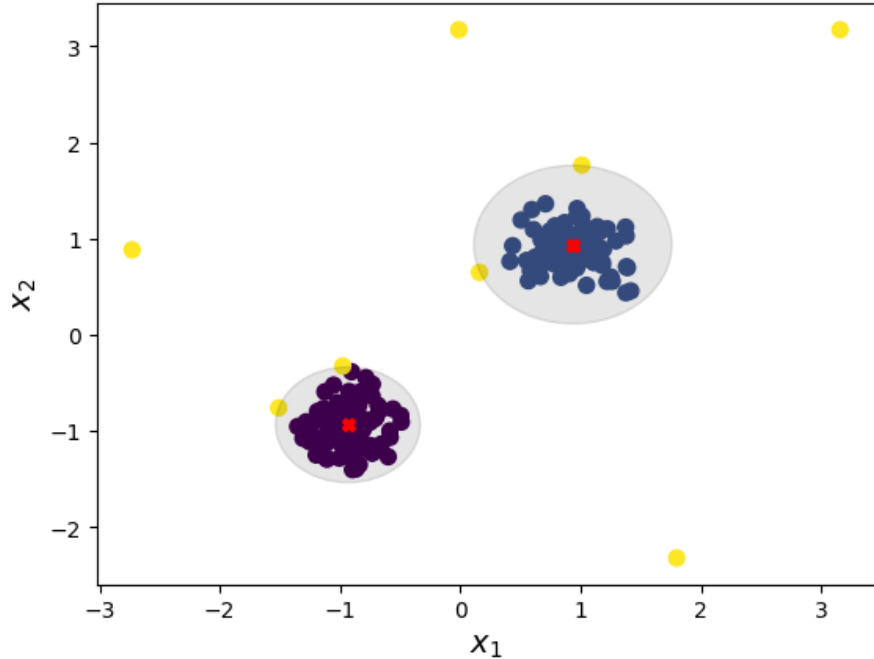


- Assim, o valor ideal de k é encontrando localizando-se o cotovelo na curva.
- Quando o cotovelo não é visível (transição suave), uma técnica comum é traçar uma reta conectando os pontos extremos da curva e identificar o ponto mais distante dessa reta como o "cotovelo" ideal.

Outros algoritmos de *clustering*

- DBSCAN: clusterização baseada em densidade.
 - *Clusters* são regiões **densas** separadas por regiões vazias.
- HDBSCAN: Extensão do DBSCAN que constrói uma **hierarquia de densidades**.
- *Gaussian mixtures*: Assume que os dados vêm de uma mistura de **distribuições gaussianas**.
- Documentação da biblioteca *scikit-learn* sobre clusterização:
 - <https://scikit-learn.org/stable/modules/clustering.html>

Aplicação: detecção de anomalias



- Nessa aplicação, encontramos os centroides e a área dos *clusters*.
- A área é definida por um valor de raio que faça com que 97% das amostras tenham distância ao centroide mais próximo menor ou igual a esse valor.
- Uma amostra é classificada como anomalia se a distância entre ela e o centroide mais próximo for significativamente alta.
 - Ou seja, se ela estiver fora da área do *cluster*.

Exemplo

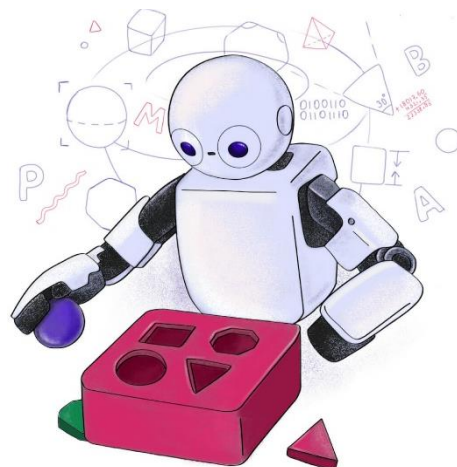
- [Exemplo: kmeans.ipynb](#)



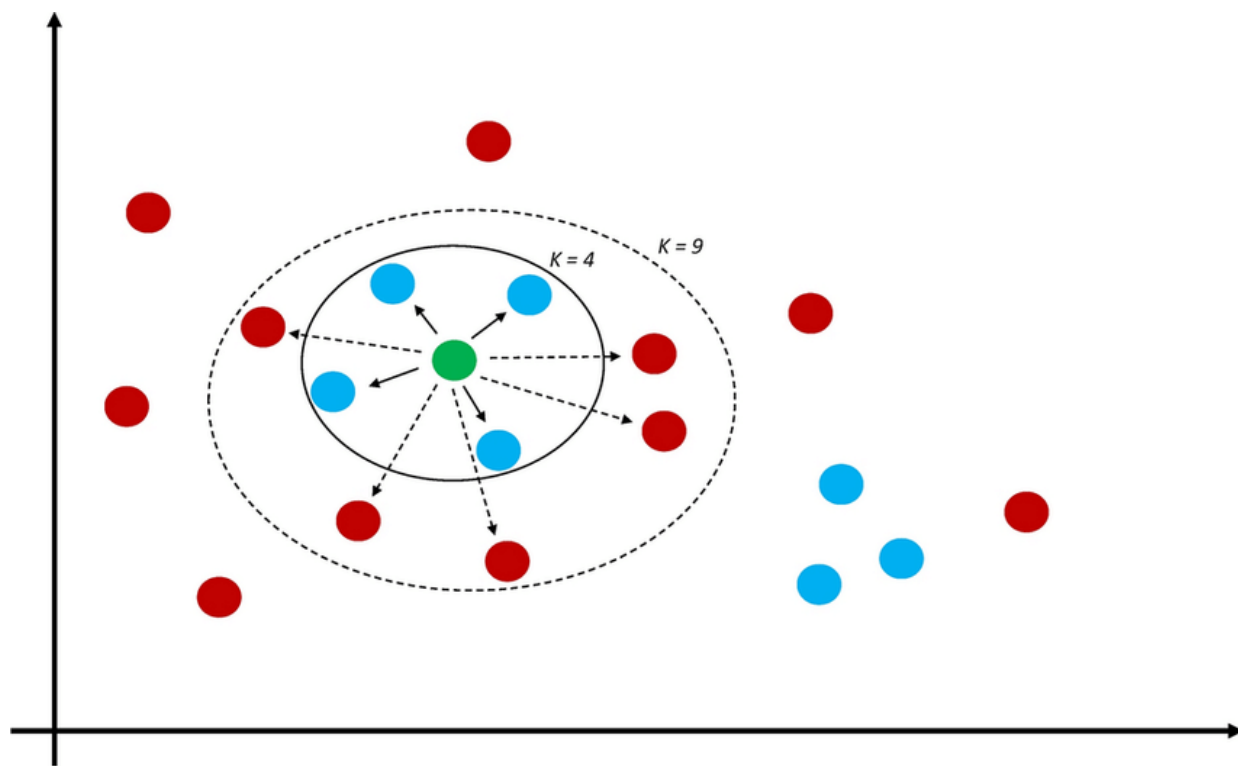
Lista de exercícios

Lista #13 – kmeans

C24 – Inteligência Artificial: *k-vizinhos mais próximos*

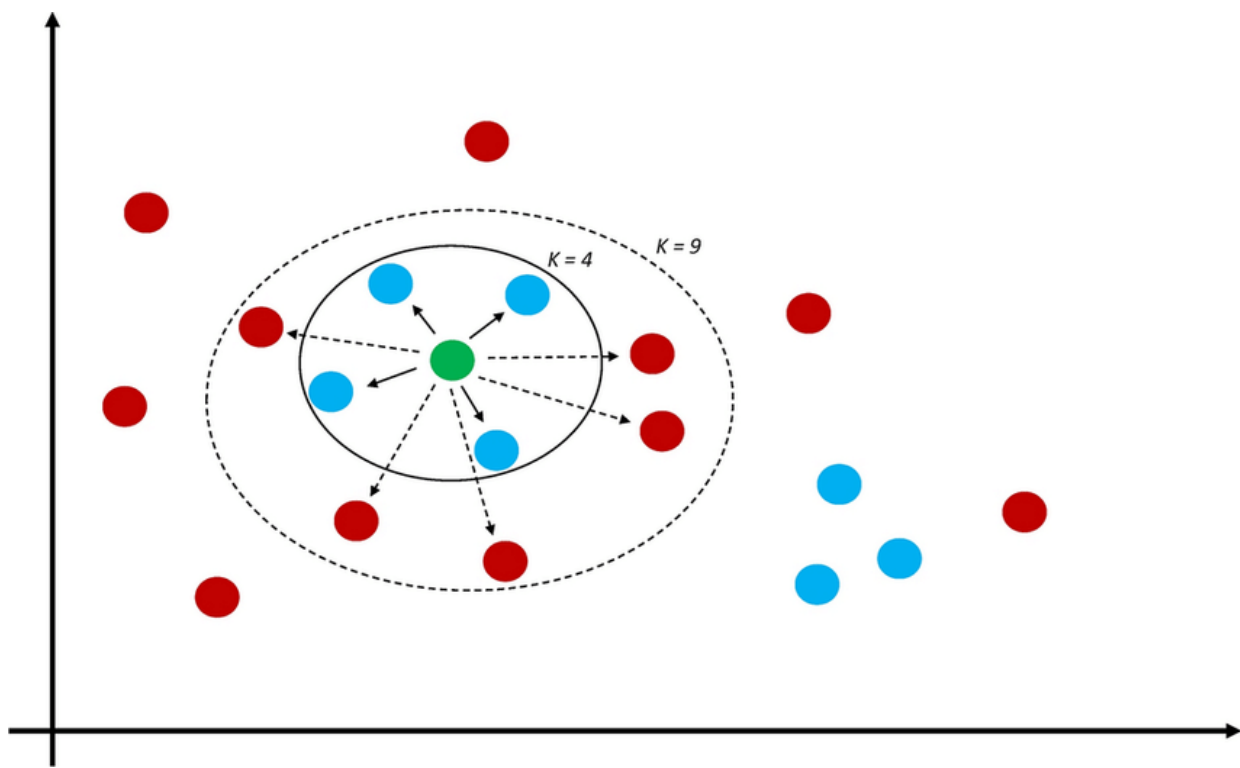


k-vizinhos mais próximos (k-NN)



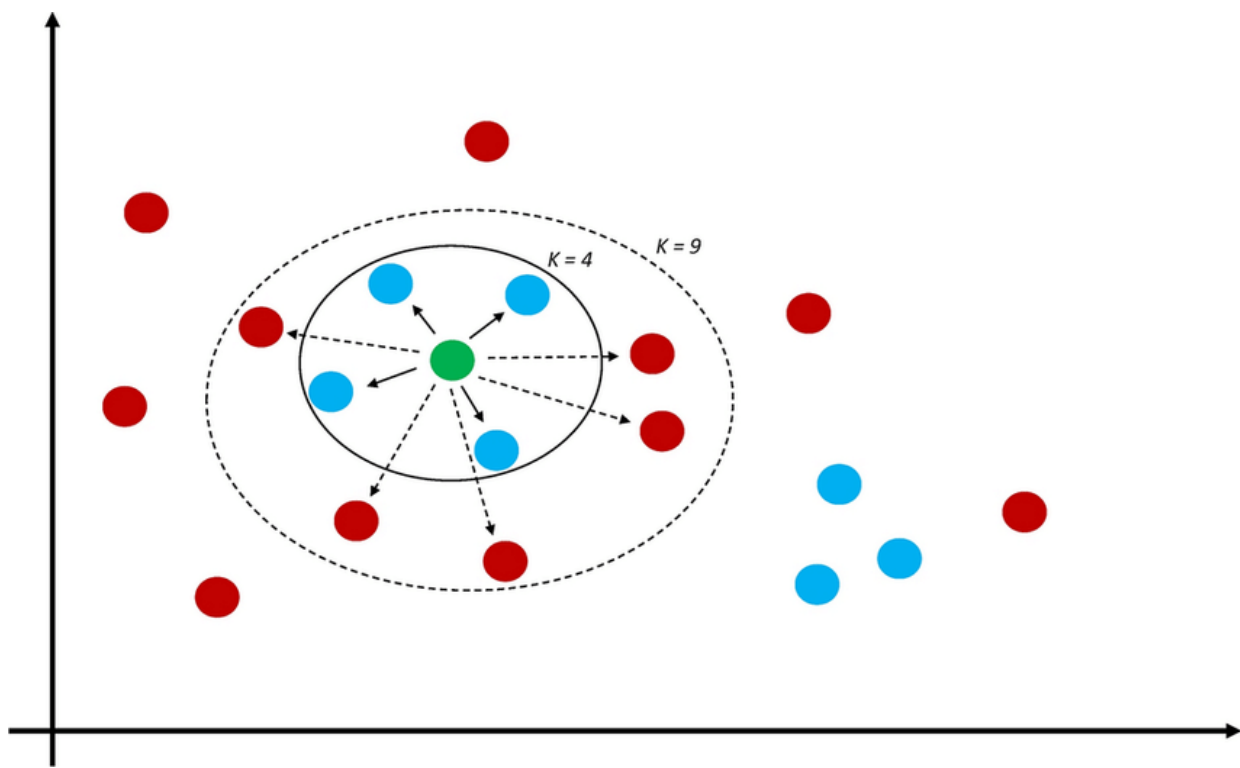
- O algoritmo k-NN (do inglês, *k-Nearest Neighbours*) é um dos algoritmos mais simples de **aprendizado supervisionado** para se resolver problemas tanto de **classificação** quanto de **regressão**.

k-vizinhos mais próximos (k-NN)



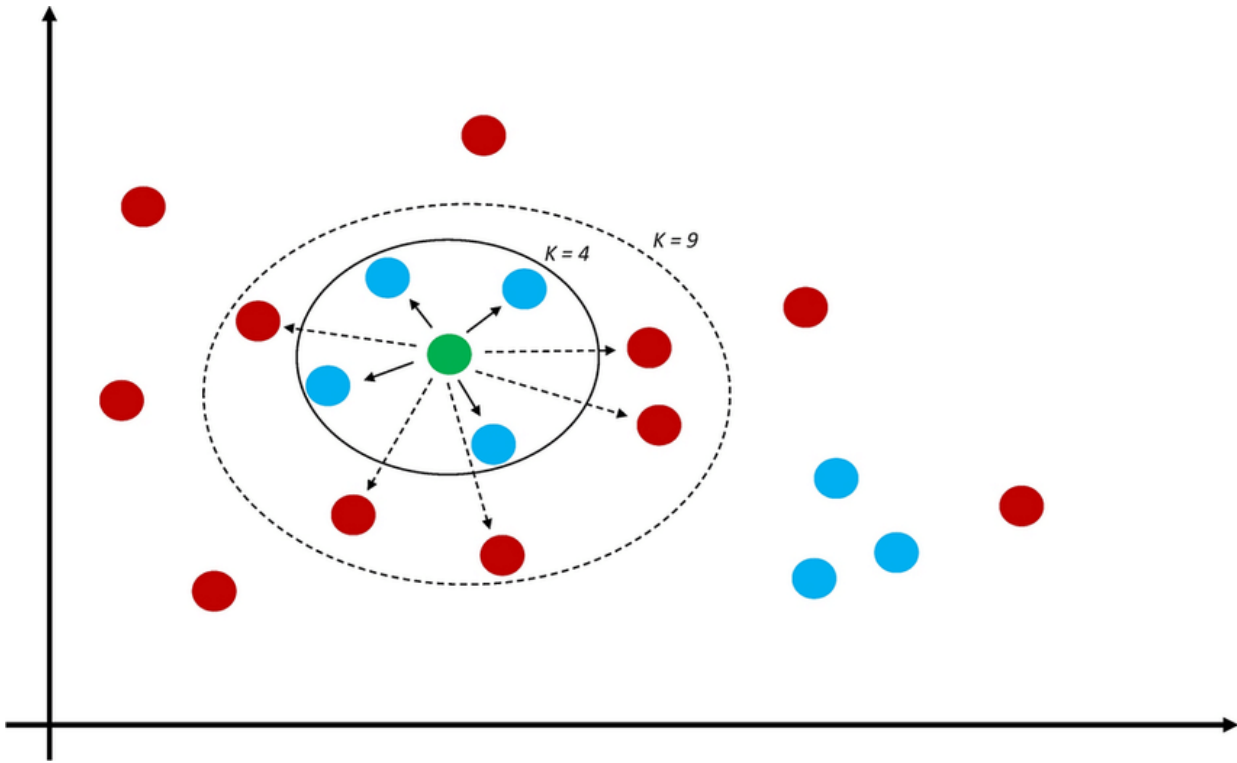
- É um algoritmo do tipo **não-paramétrico**, pois diferentemente dos outros algoritmos que vimos até o momento
 - **não há um modelo a ser treinado** (e.g., polinômio com um número definido de pesos - Regressão/Classificação logística),
 - **tampouco se faz qualquer suposição a respeito da distribuição dos dados** (e.g., modelos Naive Bayes - Bernoulli ou Gaussiano).

k-vizinhos mais próximos (k-NN)



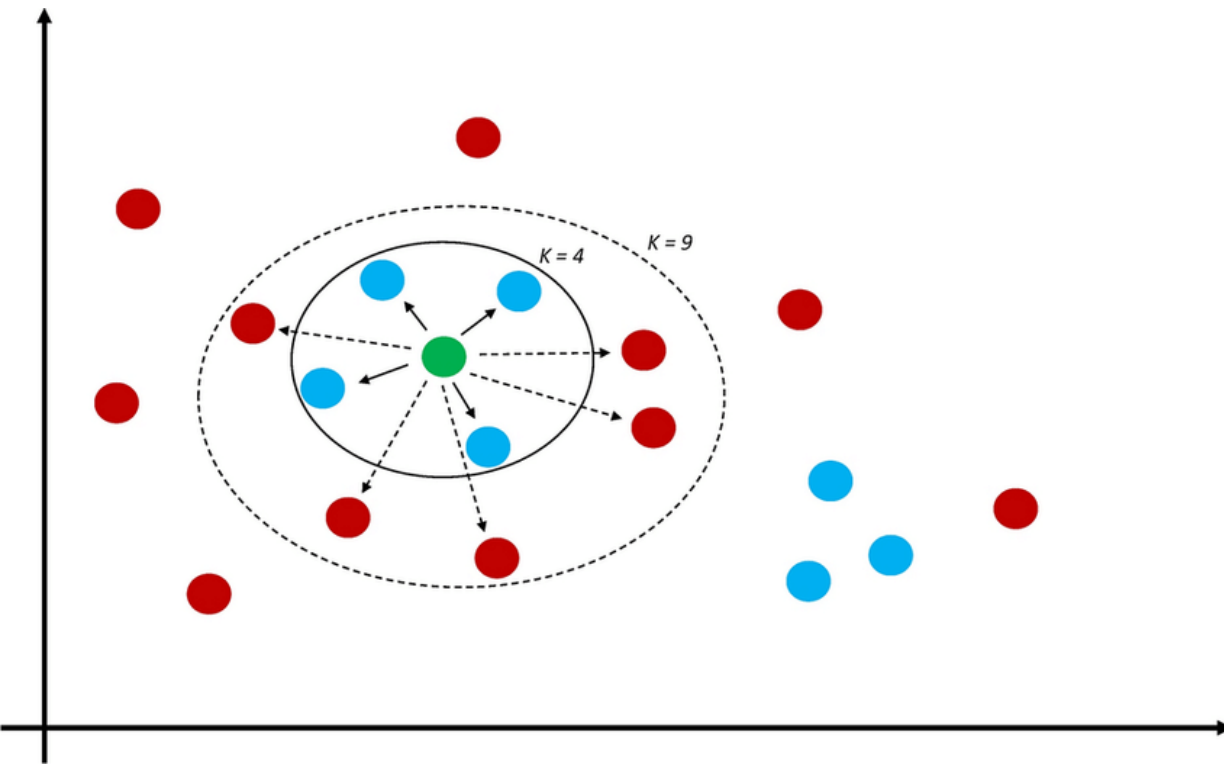
- Ele *usa diretamente os exemplos de treinamento* para tomar decisões.
- A única suposição é que uma *medida de distância entre dois exemplos* (i.e., vetores de atributos) *possa ser calculada*.

Funcionamento



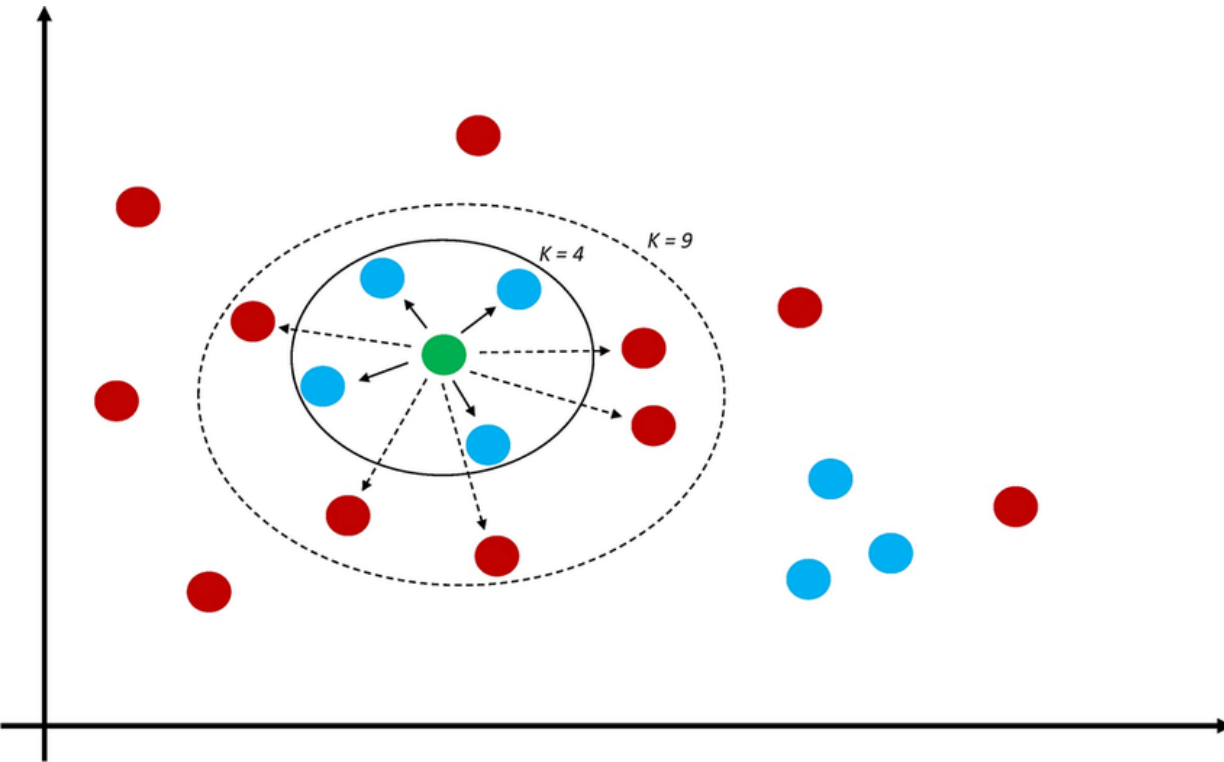
- O algoritmo necessita que todos os *exemplos de treinamento*, $\mathbf{x}(i) = [x_1(i) \quad \cdots \quad x_K(i)] \in \mathbb{R}^{K \times 1}$ e seus respectivos rótulos, $y(i)$, $i = 0, \dots, N - 1$, sejam *armazenados em memória*.

Funcionamento



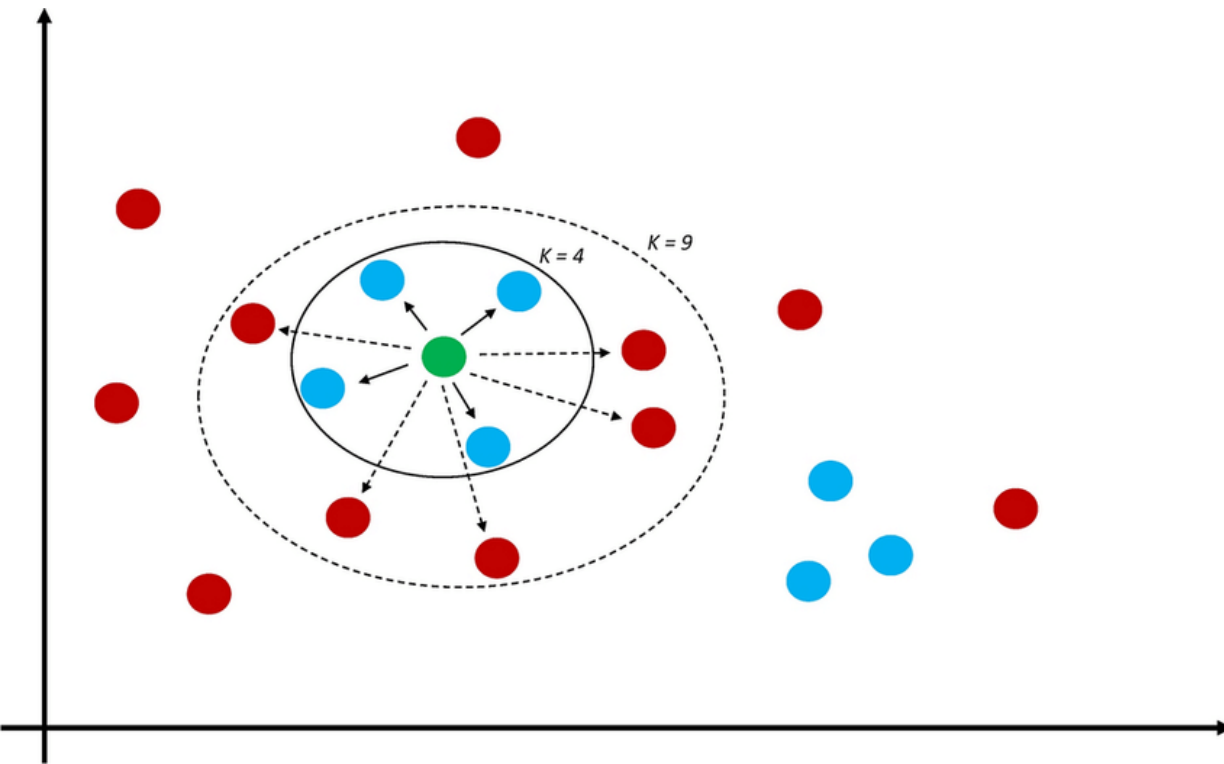
- Em seguida, dado um **exemplo de entrada inédito**, x' (e.g., círculo verde), a saída para este exemplo **dependerá dos rótulos** associados aos **k exemplos de treinamento mais próximos** do exemplo de entrada x' no espaço de atributos.

Funcionamento



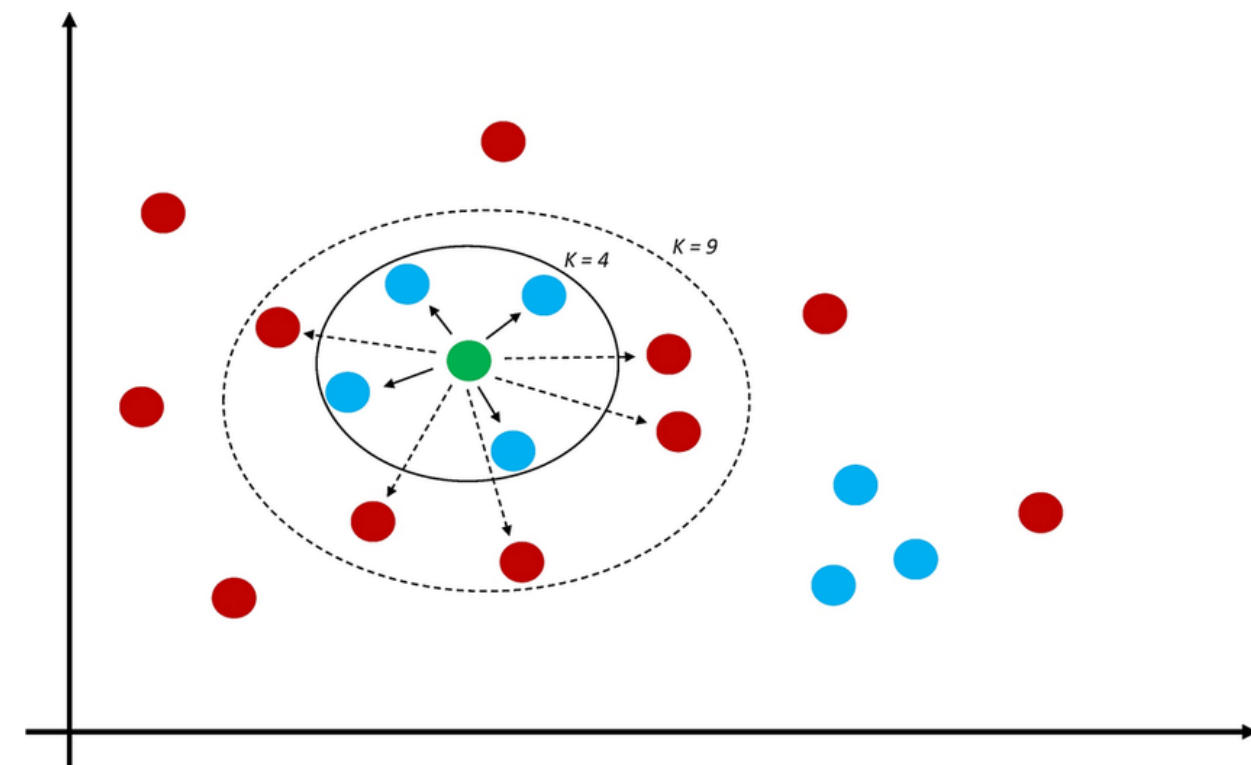
- Para *classificação*, a *classe mais frequente* entre os k vizinhos mais próximos é escolhida como a classe do exemplo de entrada.
- Para *regressão*, os valores associados aos k vizinhos mais próximos são usados para calcular um *valor médio ou ponderado*, que será a estimativa para o exemplo de entrada.

Funcionamento



- Ou seja, o algoritmo usa *similaridade/proximidade entre vetores de atributos* para prever os valores de quaisquer novos exemplos.
- Isso significa que um novo exemplo de entrada recebe um valor de saída com base na sua *proximidade com os exemplos do conjunto de treinamento*.

Funcionamento

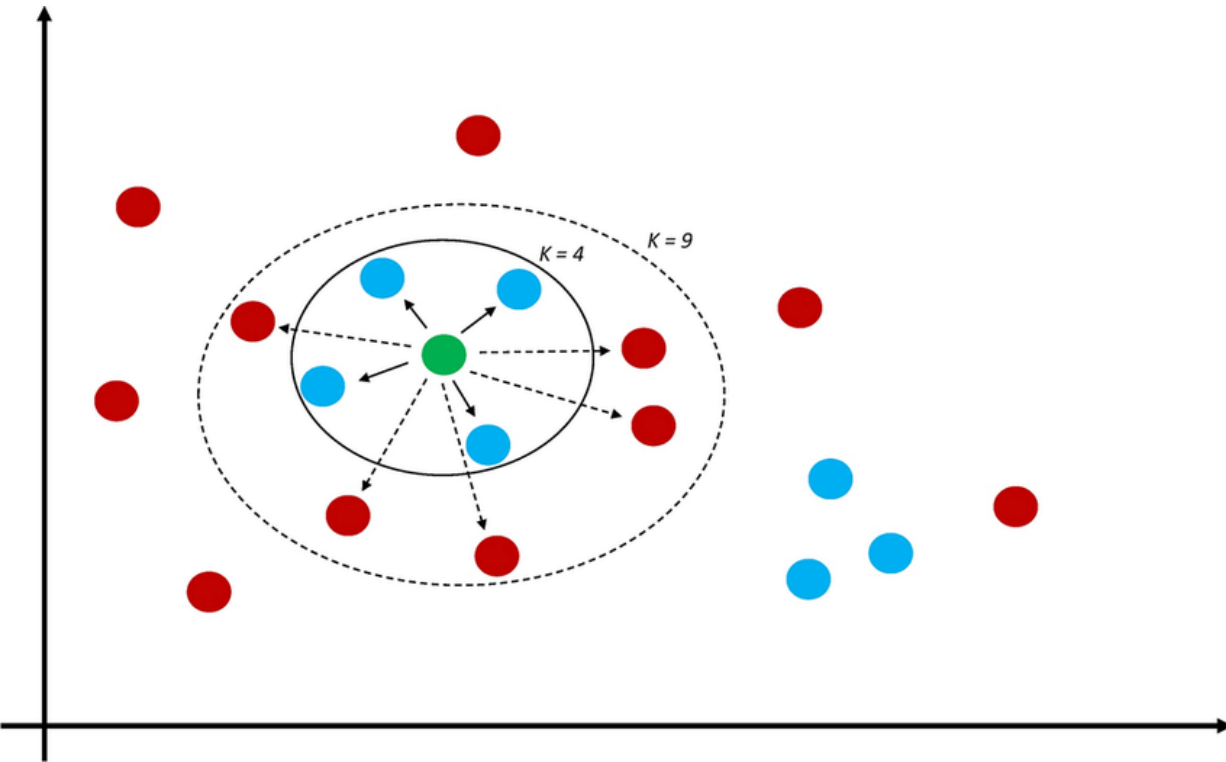


- Por exemplo, para regressão, nós podemos tomar a **média aritmética dos rótulos dos k vizinhos mais próximos**:

$$\hat{y}(x') = \frac{1}{k} \sum_{x(i) \in N_k(x')} y(i),$$

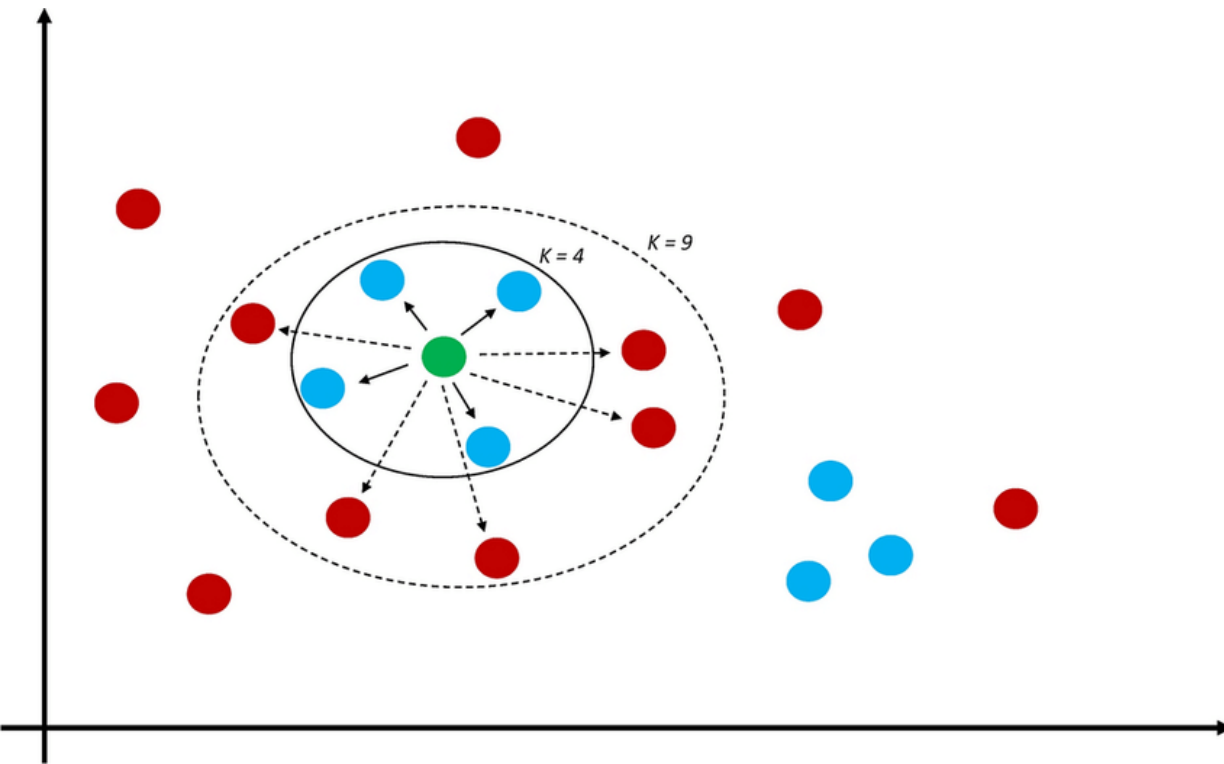
onde $N_k(x')$ é a vizinhança de x' , formada pelos exemplos de treinamento $x(i)$ que correspondem aos k vizinhos mais próximos de x' .

Funcionamento



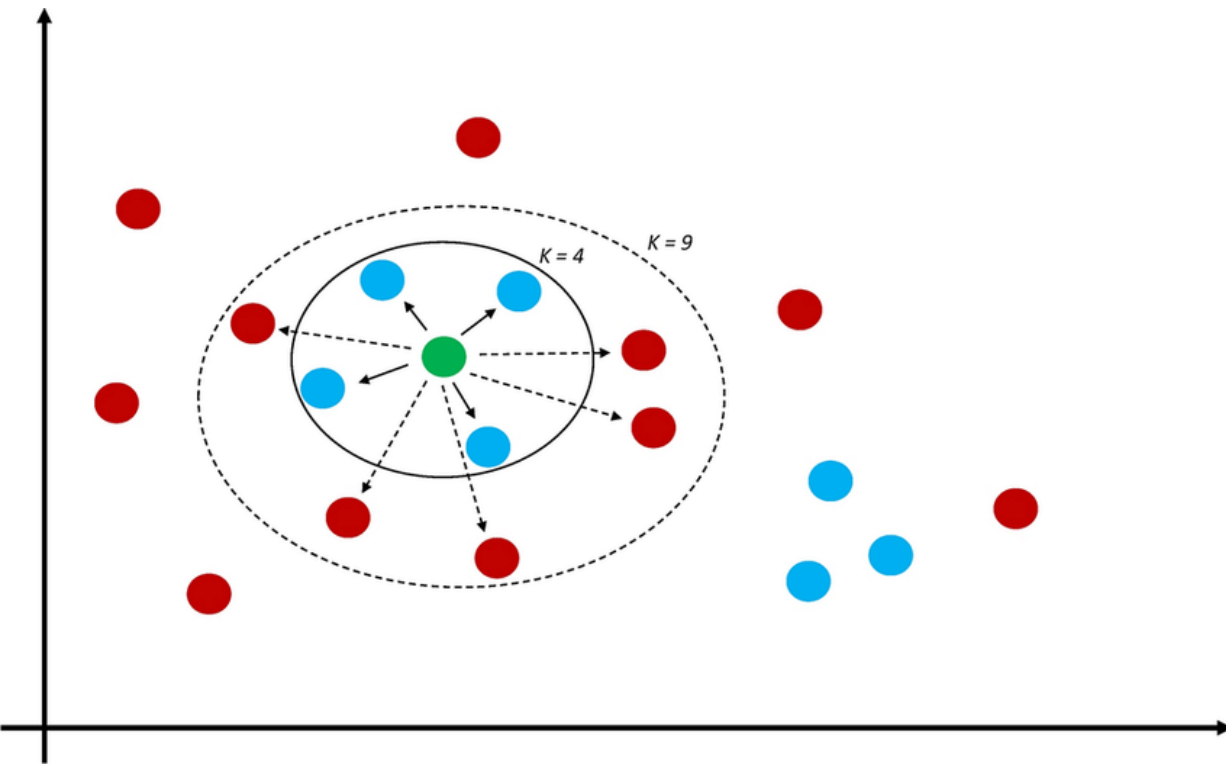
- Para classificação, por exemplo, dentre k vizinhos mais próximos, escolhemos a classe com maior número de exemplos (i.e., voto majoritário).
- **OBS.:** Não confundam o *número de atributos*, K , com o *número de vizinhos mais próximos*, k .

k-Vizinhos mais Próximos (k-NN)



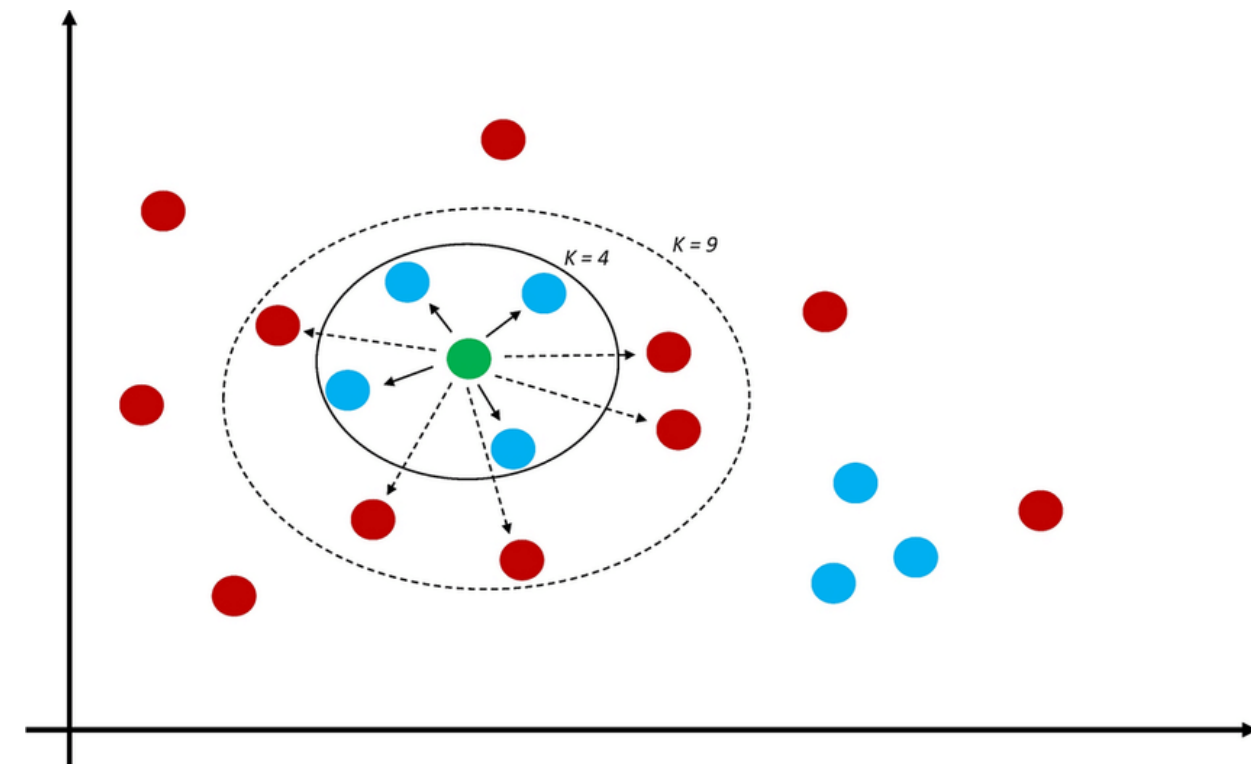
- Portanto, o uso do k-NN envolve a **definição** de:
 - Uma **métrica de distância** que deve ser **calculada no espaço de atributos** a fim de identificar os vizinhos mais próximos.
 - Um **valor para o hiperparâmetro k** , ou seja, a escolha do número de vizinhos que devem ser levados em consideração para a geração da saída correspondente ao exemplo de entrada, x' .

k-Vizinhos mais Próximos (k-NN)



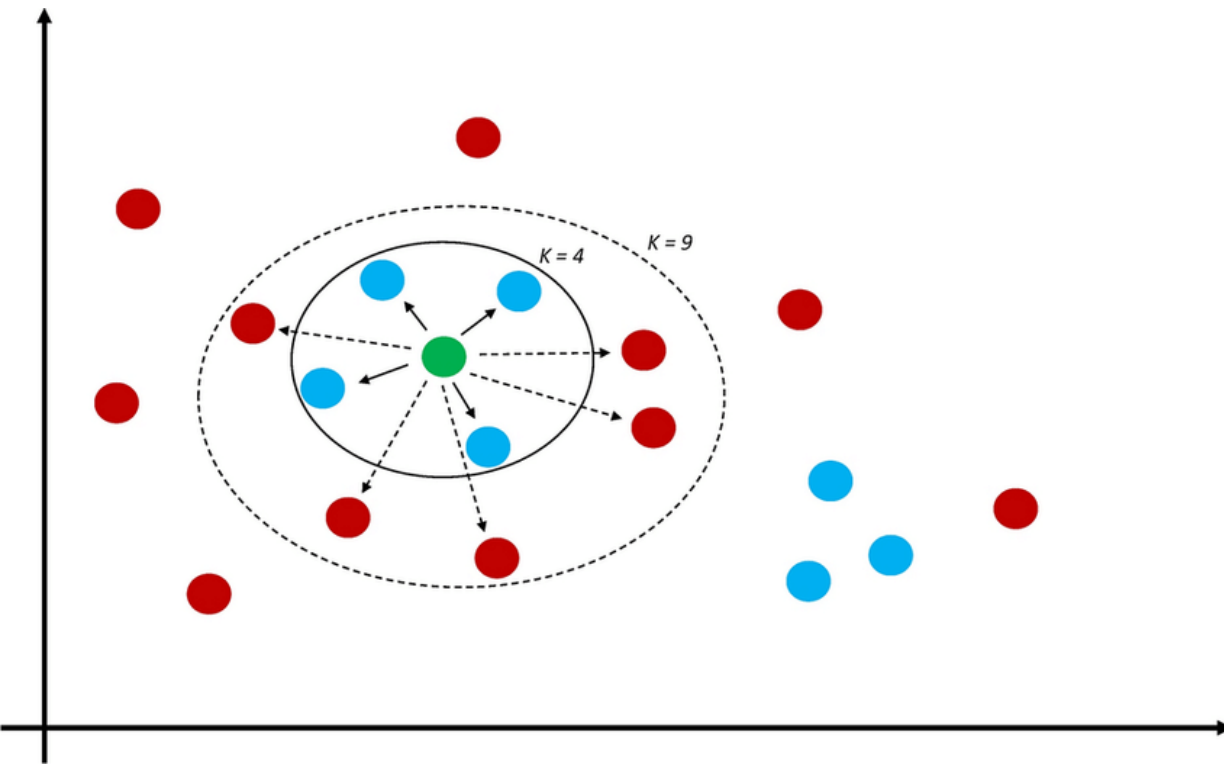
- Como k é um **hiperparâmetro** do k-NN, devemos usar técnicas de **validação cruzada** para encontrar o melhor valor de k .
- Podemos utilizar por exemplo **Grid Search** ou **Random Search com k-fold**.

k-Vizinhos mais Próximos (k-NN)



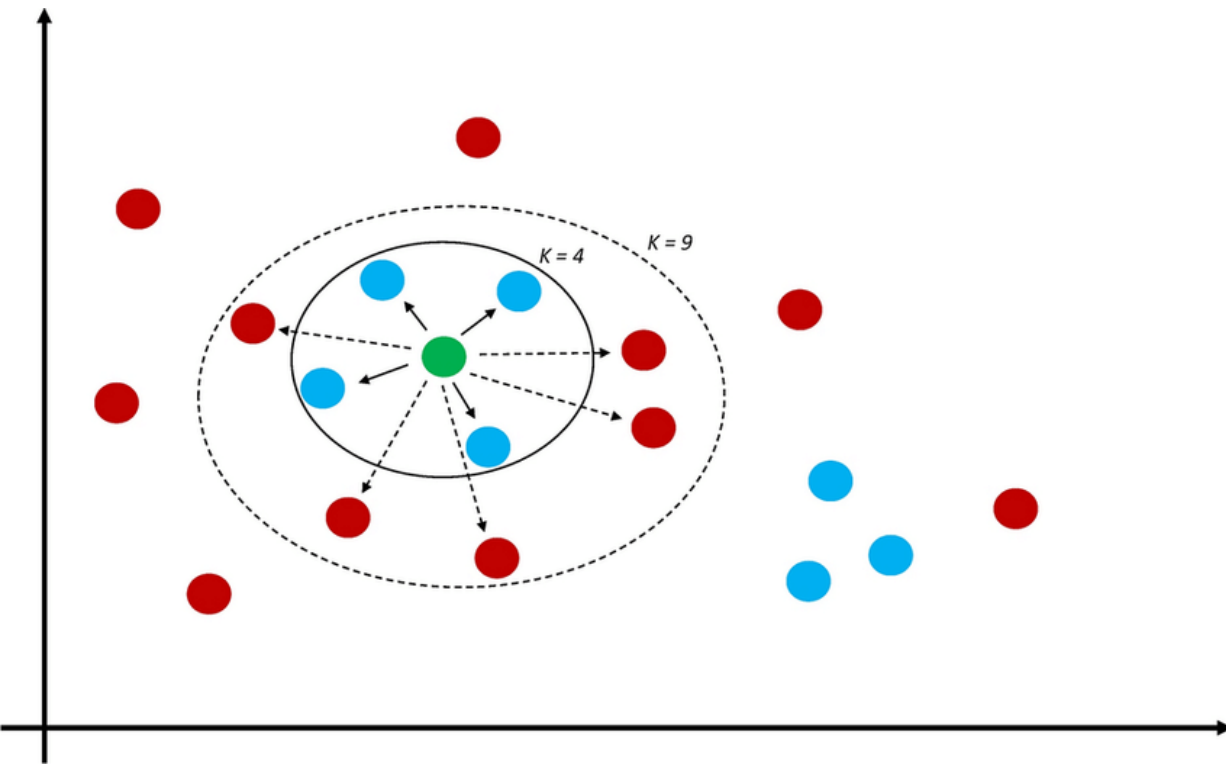
- Devido às suas características, o k-NN é visto como um **algoritmo de aprendizado competitivo**, uma vez que os **elementos do modelo** (que são os próprios exemplos de treinamento) **competem entre si pelo direito de influenciar a saída do algoritmo** quando a **medida de similaridade (distância)** é calculada para cada novo dado de entrada.

k-Vizinhos mais Próximos (k-NN)



- O k-NN explora a ideia conhecida como *lazy learning*, uma vez que o algoritmo *não “constrói” um modelo até o instante em que uma predição é solicitada*.
 - Ou seja, *não existe uma etapa explícita de treinamento/aprendizado*.
 - Em vez disso, todo *o aprendizado é adiado até a fase de predição*.

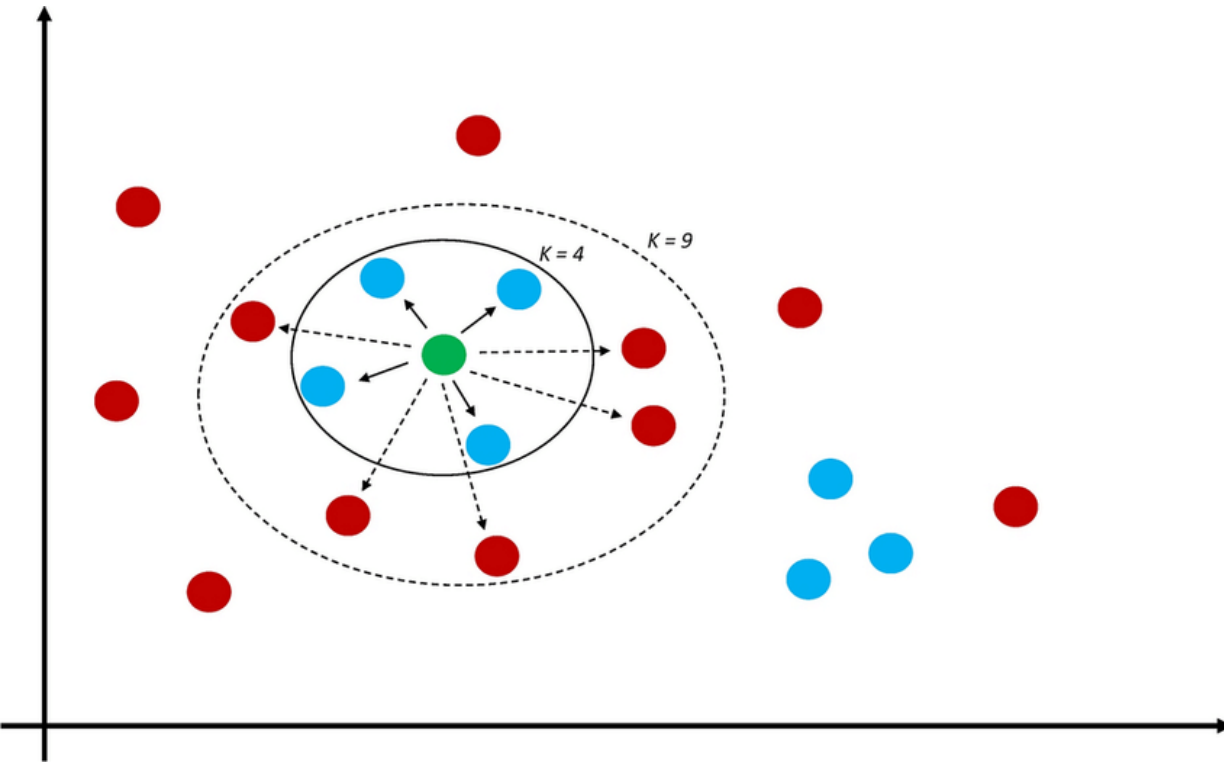
k-Vizinhos mais Próximos (k-NN)



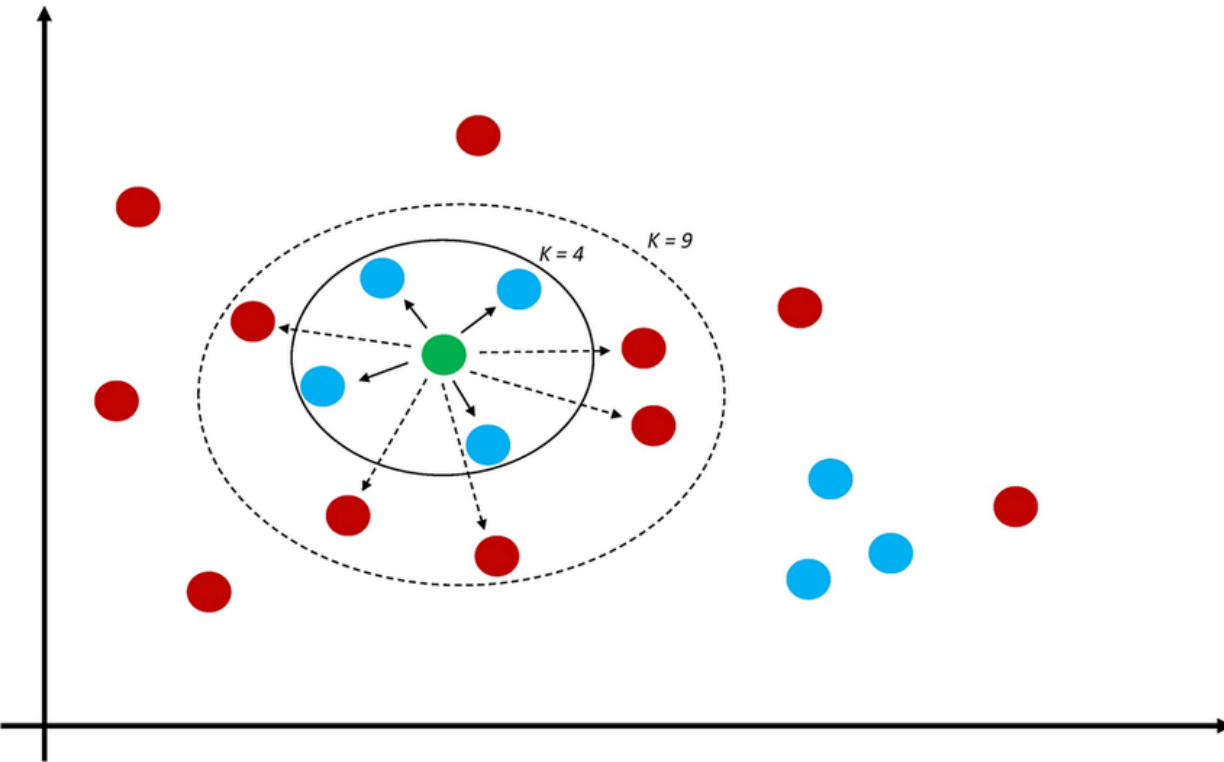
- O k-NN segue o paradigma de ***aprendizado-baseado em exemplos***, onde ao invés de se treinar um modelo a partir do conjunto de treinamento, ele ***compara exemplos de entrada com os exemplos do conjunto de treinamento armazenados em memória***.

Desvantagens

- Todos os dados de treinamento precisam ser *armazenados* e *consultados* para se identificar os k vizinhos mais próximos.

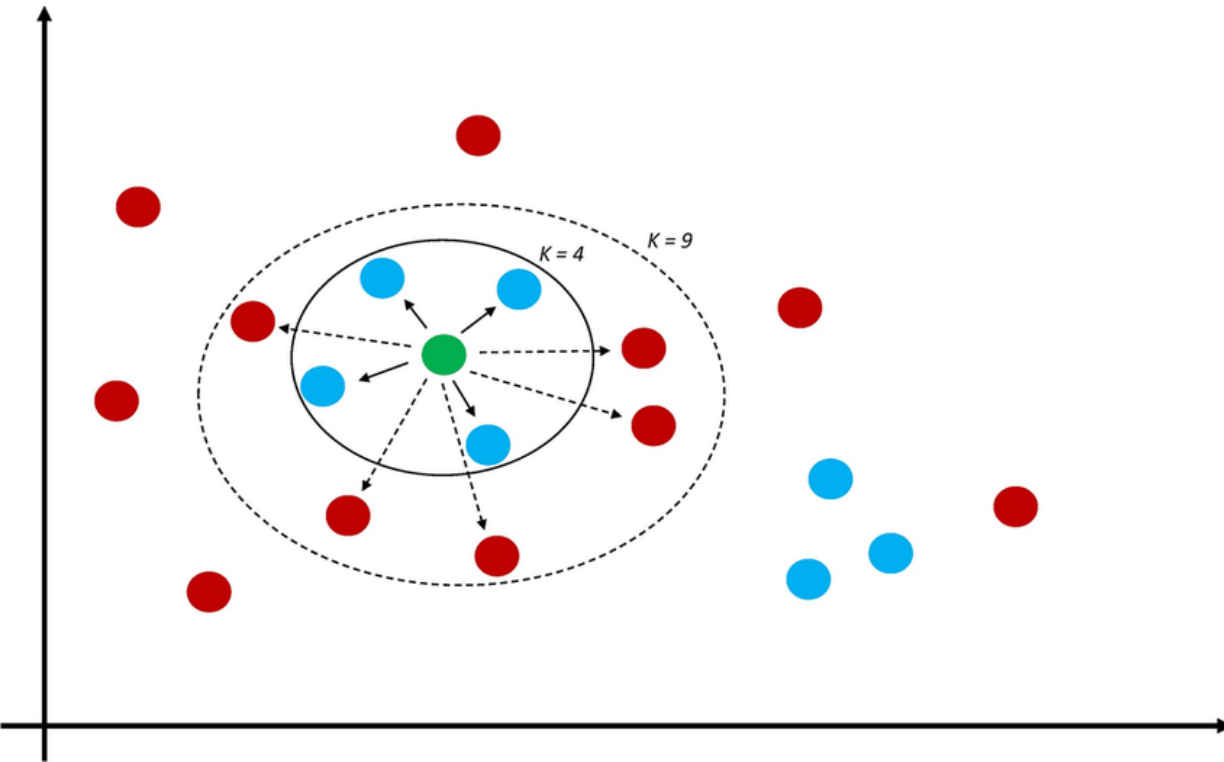


Desvantagens



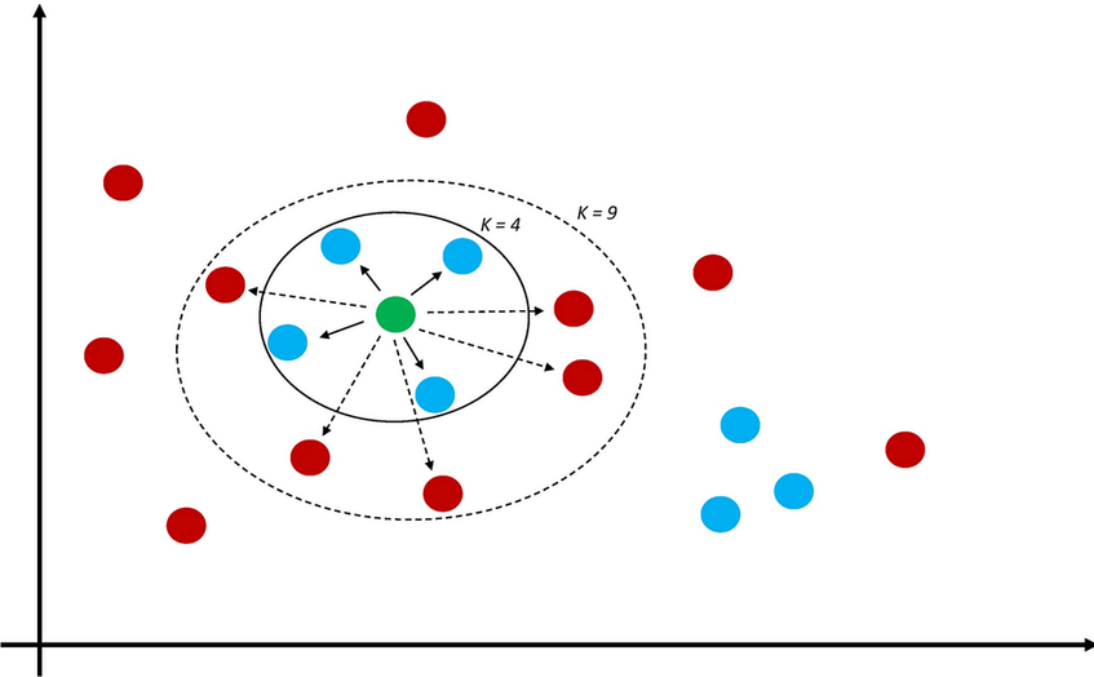
- Portanto, a ***predição*** poderá ser ***demorada*** dependendo do tamanho do conjunto de treinamento, pois deve-se ***calcular a distância entre o exemplo de entrada e todos os exemplos do conjunto de treinamento***.

Desvantagens



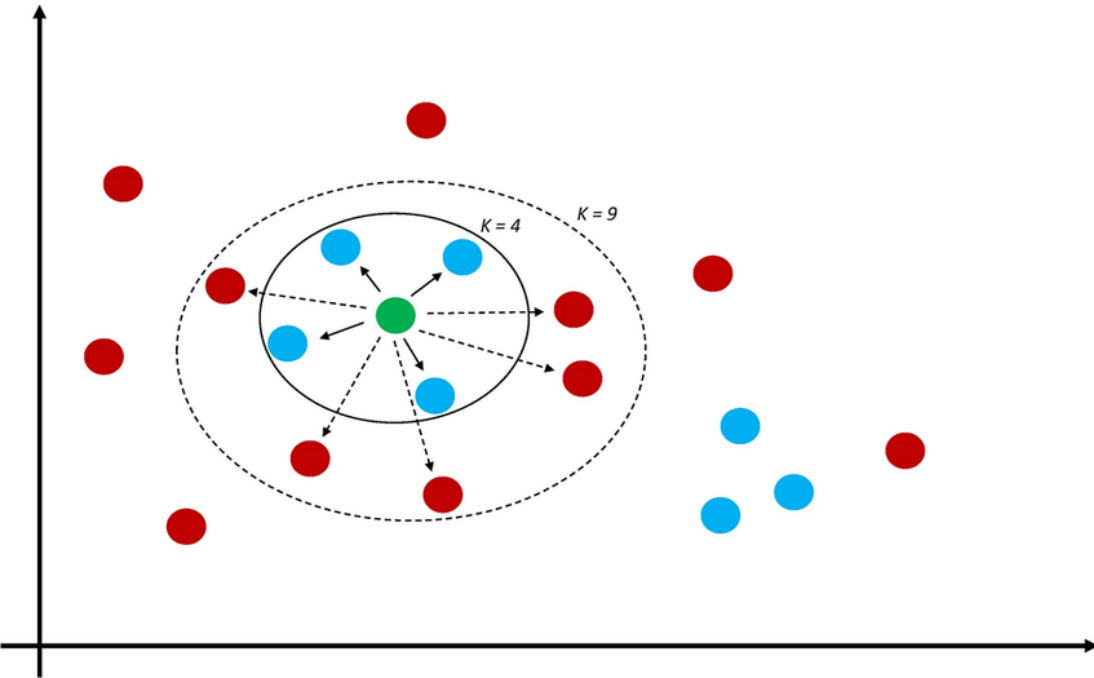
- Além disto, como vimos, o **conjunto de treinamento deve ser armazenado em memória**, e caso esse conjunto seja muito grande, pode não haver memória o suficiente para armazená-lo.

Outras desvantagens: Sensibilidade à dimensionalidade



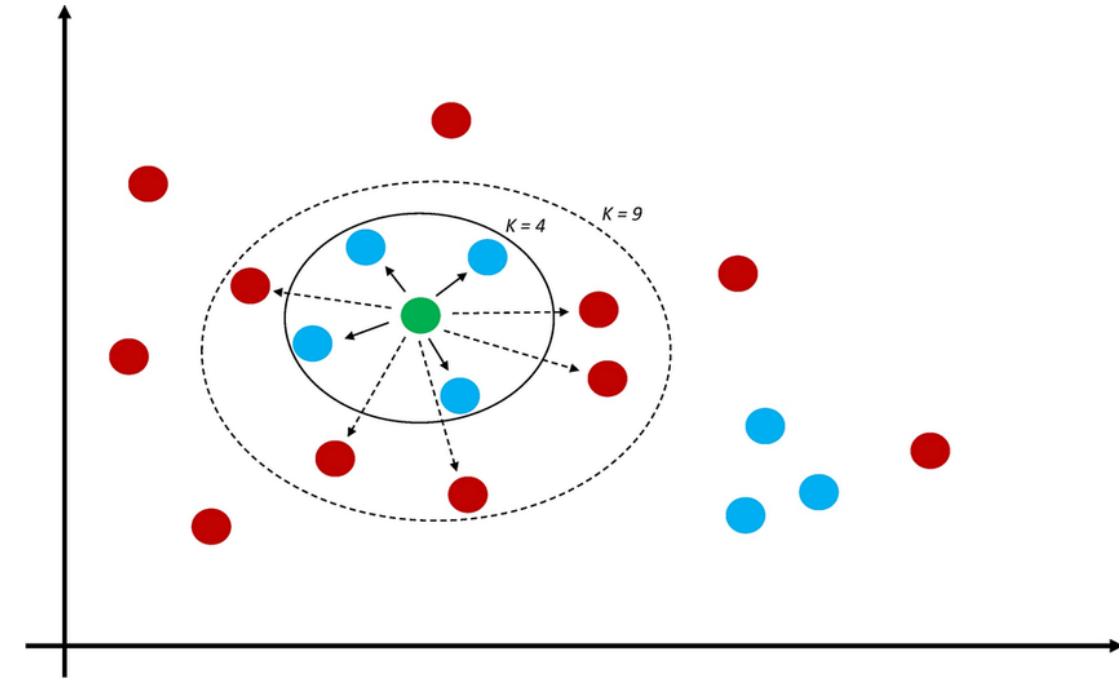
- O desempenho do k-NN pode ser afetado quando o conjunto de dados possui *alta dimensionalidade*.
- Isso ocorre porque, em espaços de alta dimensão, *a noção de proximidade entre exemplos pode se tornar menos significativa, levando a resultados menos precisos*.

Outras desvantagens: Sensibilidade à dimensionalidade



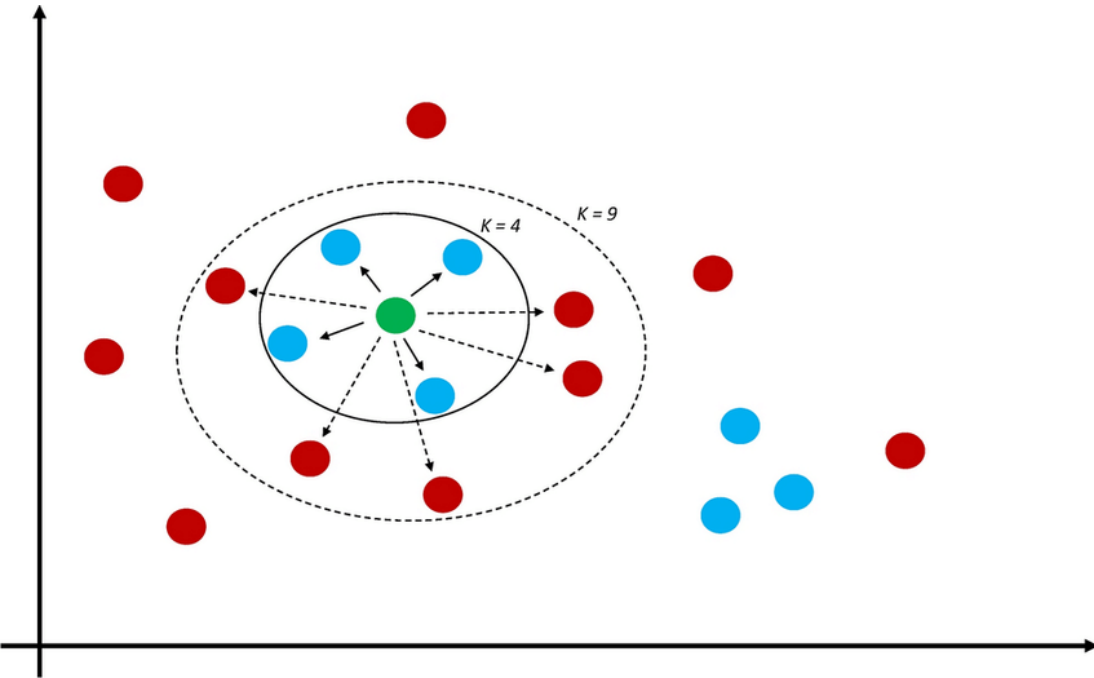
- À medida que as dimensões aumentam, os dados tornam-se mais **esparsos**, dificultando a busca por vizinhos próximos significativos.
 - **Esparsidade**: exemplos ficam mais distantes uns dos outros.
- A esparsidade faz com que existam **menos pontos vizinhos dentro de uma determinada distância**.

Outras desvantagens: Sensibilidade à dimensionalidade



- Além disso, em *espaços de alta dimensionalidade* as *distâncias* entre pontos *se tornam mais uniformes* (i.e., *equidistante*), o que *reduz a relevância do conceito de vizinhança*.
- Isso degrada o desempenho do k-NN, pois sua *suposição de que pontos próximos têm rótulos semelhantes pode não se aplicar* em alta dimensionalidade.
- Além disso, o *cálculo da distância se torna mais custoso computacionalmente*.

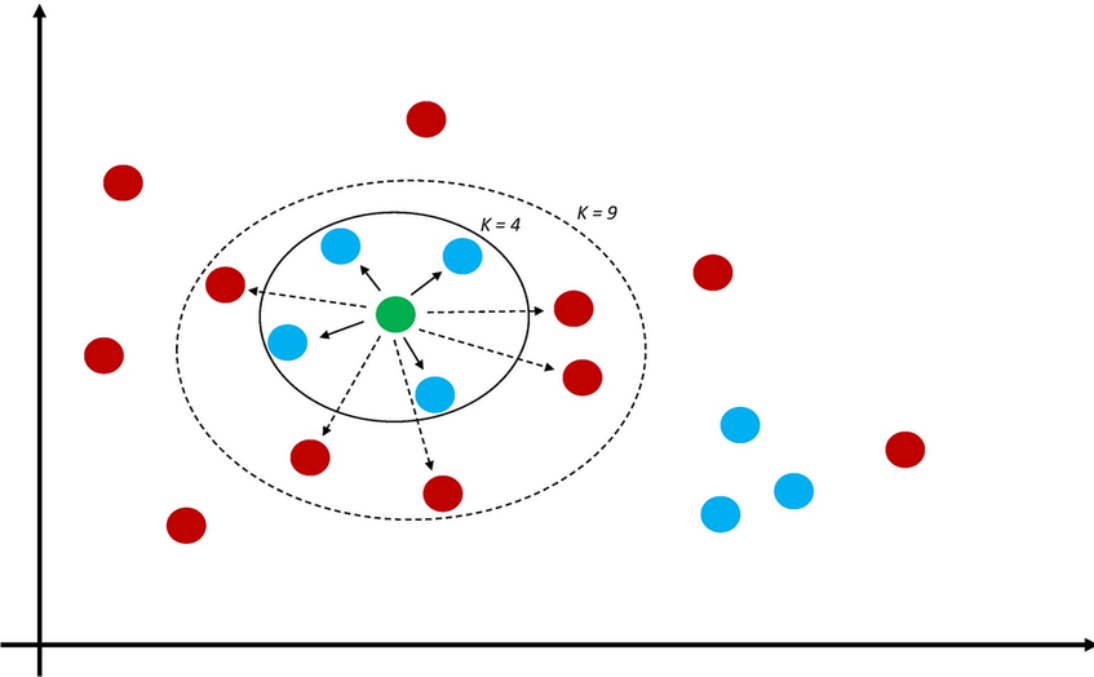
Outras desvantagens: Sensibilidade a dados desbalanceados



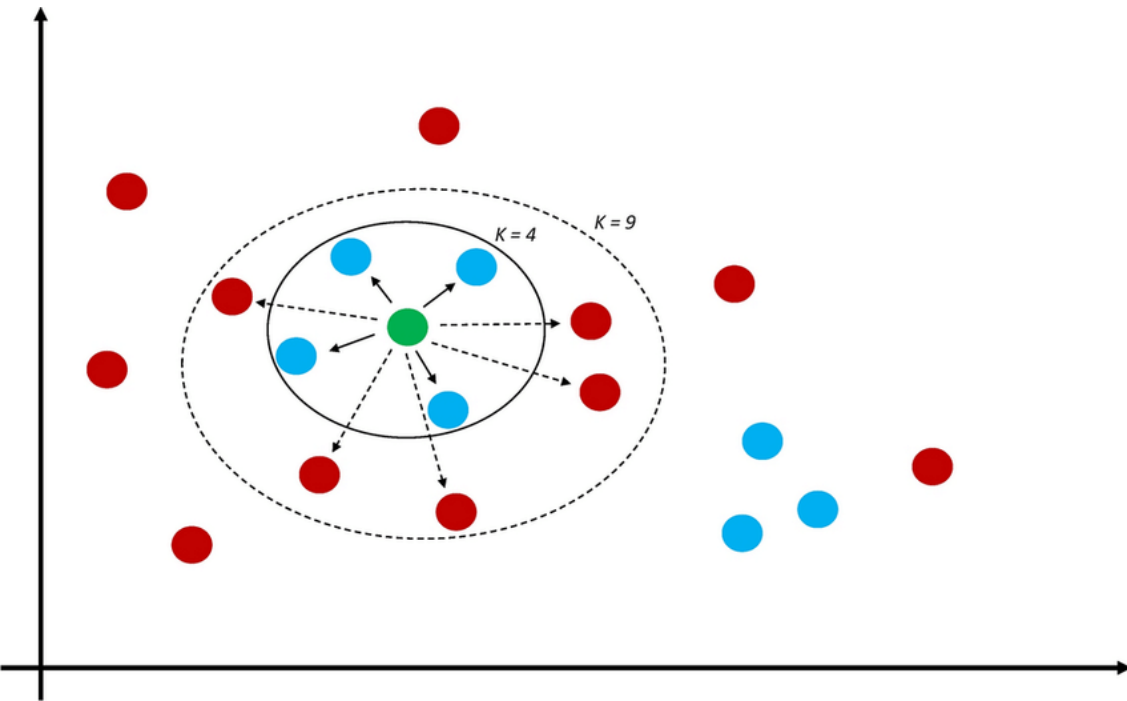
- Se as classes estiverem desbalanceadas, o k-NN pode se tornar *enviesado em direção à classe majoritária*.
- Isso ocorre porque, ao selecionar os k vizinhos mais próximos, é mais provável que sejam selecionados exemplos da classe majoritária.

Outras desvantagens: Necessidade de pré-processamento de dados

- O k-NN é *sensível a atributos com diferentes escalas*.
- Portanto, em geral, *é necessário realizar escalonamento*, como normalização ou padronização, para garantir que todos os *atributos* tenham uma *contribuição similar no cálculo da distância*.

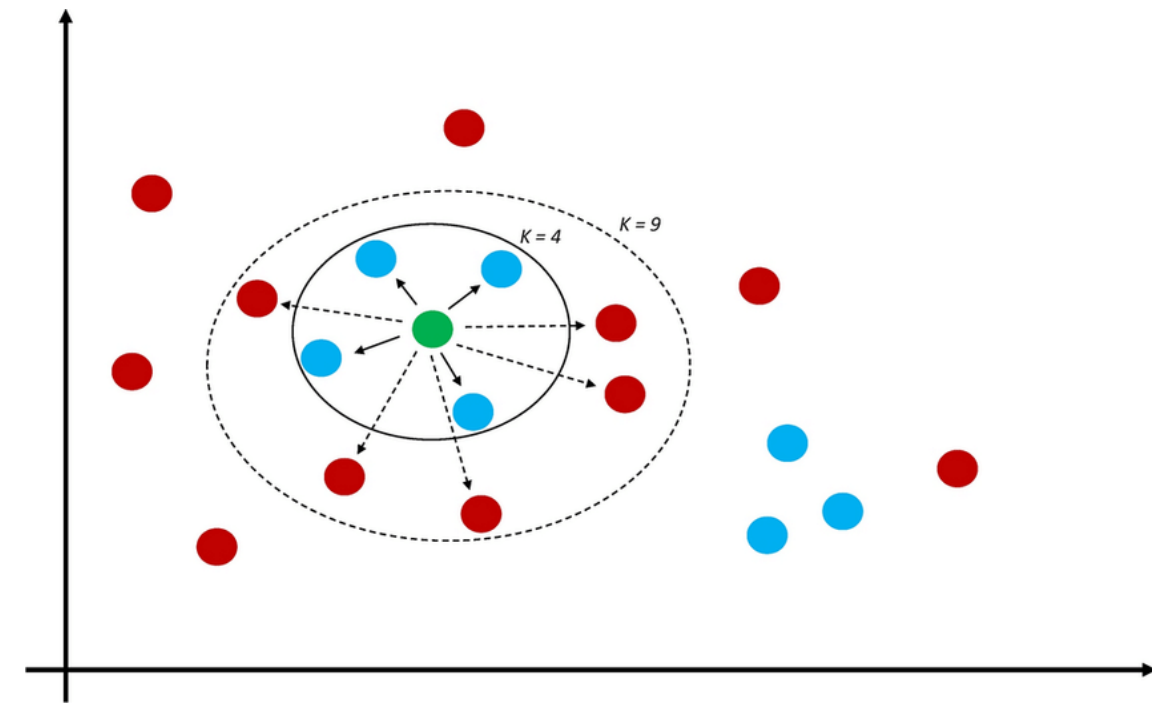


Métricas de distância



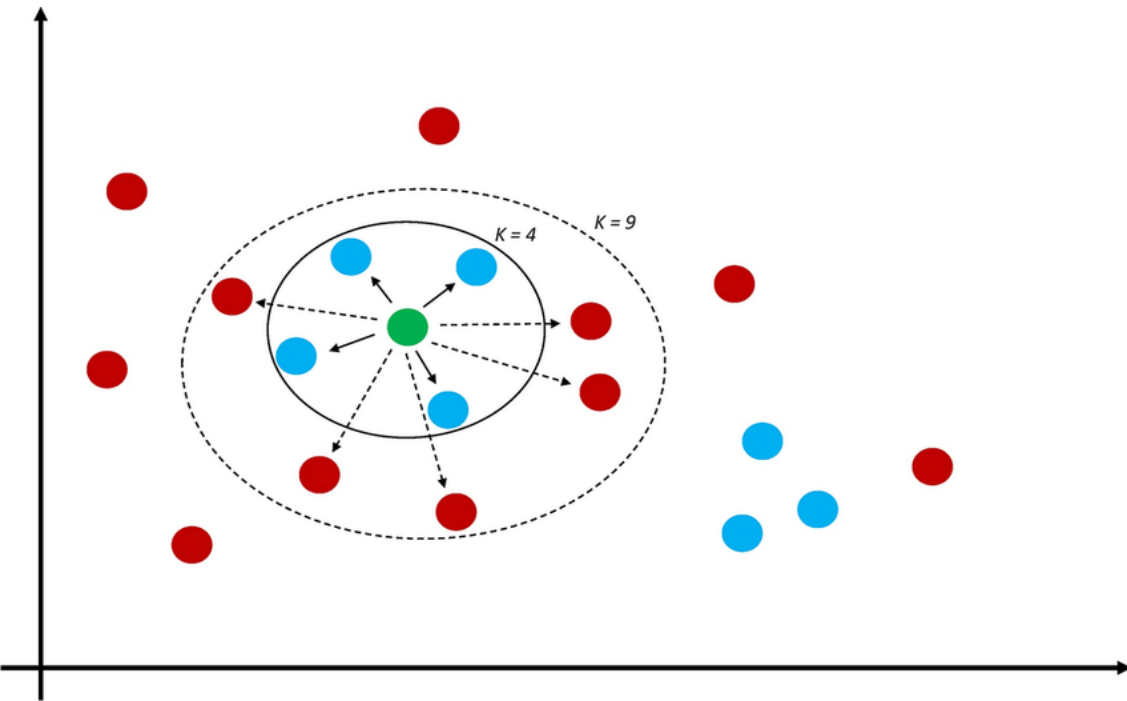
- **Definição:** Uma métrica de distância fornece a *distância entre os elementos de um conjunto*.
- Se a *distância* é igual a *zero*, os *elementos são equivalentes*, *caso contrário, os elementos são diferentes* uns dos outros.

Métricas de distância



- No nosso caso, a métrica serve para *medir a distância/similaridade* entre os *K atributos do vetor de entrada* e os *K atributos dos vetores do conjunto de treinamento*.
- Existem várias *métricas de distância*, mas vamos discutir apenas as mais utilizadas através de uma *métrica de distância generalizada*, chamada de *distância de Minkowski*.

Métricas de distância

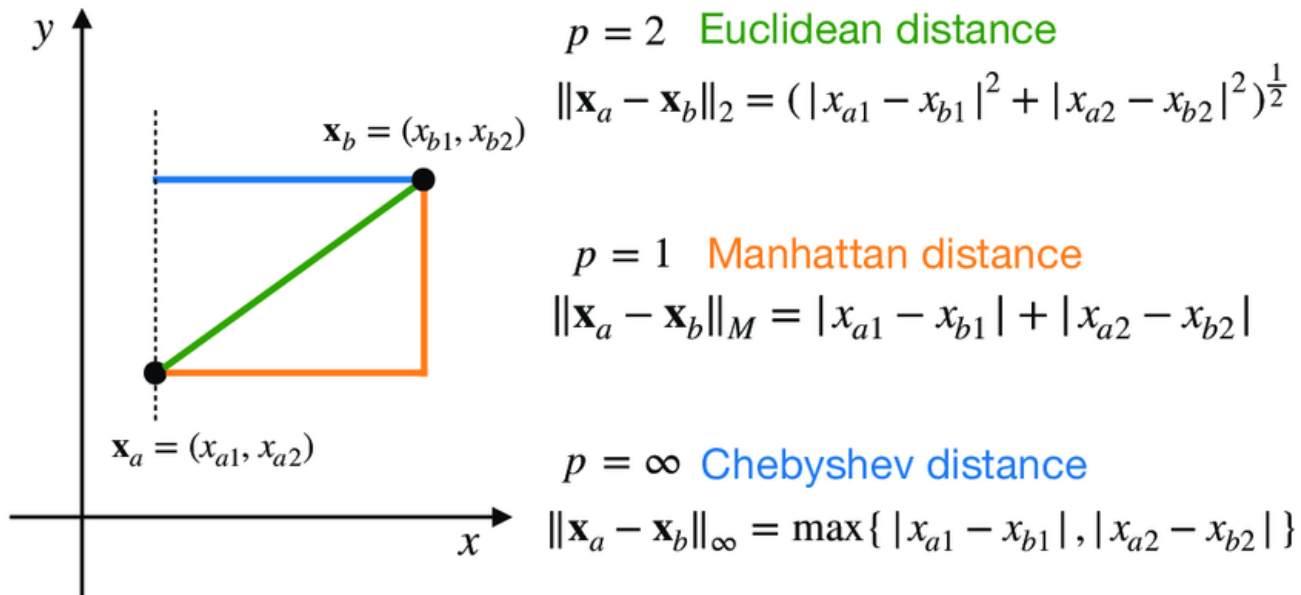


- **Distância de *Minkowski***: é uma métrica definida no ***espaço vetorial normado*** (ou seja, um ***espaço vetorial*** no qual uma ***norma vetorial***, $p(\cdot)$, é definida) que satisfaz algumas propriedades.
 - É um ***espaço onde podemos medir o tamanho ou magnitude dos vetores***.

Métricas de distância

- A **norma vetorial**, $p(\cdot)$, é uma **função que mapeia um vetor em um número real não negativo**, i.e., $\mathbb{R}^{K \times 1} \rightarrow \mathbb{R}_+$, e que possui algumas propriedades.
- Sejam 2 vetores, $\mathbf{v}$ e $\mathbf{u} \in \mathbb{R}^{K \times 1}$, a norma $p(\cdot)$ dos vetores é uma **função com valores não-negativos** com as seguintes propriedades:
 - A norma satisfaz a **desigualdade do triângulo**:
 - $p(\mathbf{v} + \mathbf{u}) \leq p(\mathbf{v}) + p(\mathbf{u})$
 - A norma é **absolutamente escalável**:
 - $p(a\mathbf{v}) = |a|p(\mathbf{v})$, para todo $a \in \mathbb{R}$
 - A norma é **positiva definida**, i.e., os valores são sempre maiores ou no mínimo iguais a zero:
 - Se $p(\mathbf{v}) = 0$, então $\mathbf{v} = \mathbf{0}$ (**vetor nulo**).

Distância de Minkowski

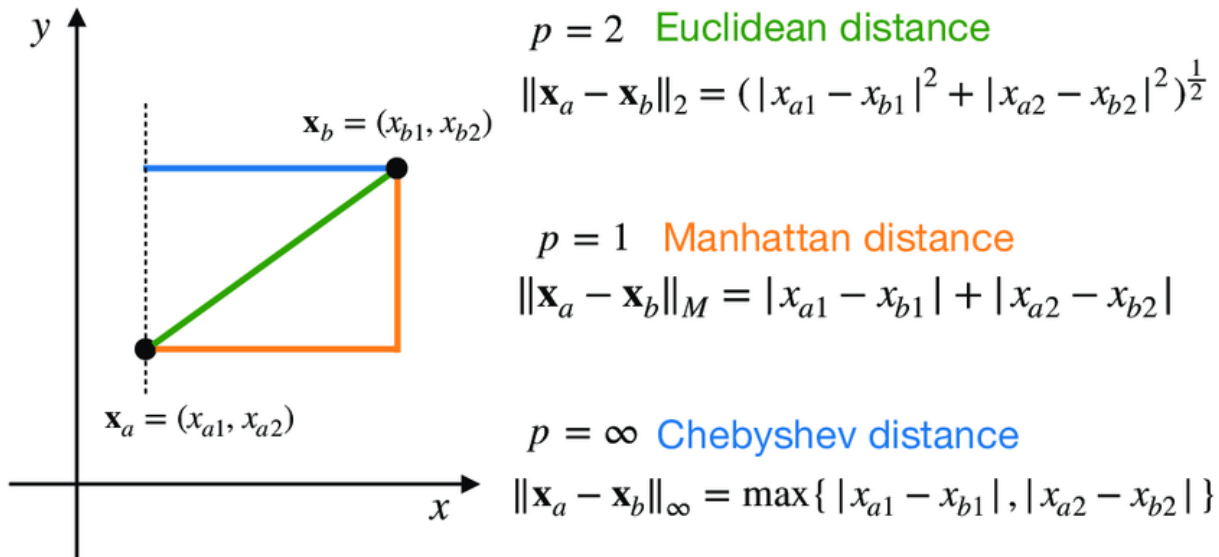


- A **distância de Minkowski** de ordem p é calculada com equação abaixo

$$d(\mathbf{x}; \mathbf{y}) = \left(\sum_{i=1}^K |x_i - y_i|^p \right)^{1/p}.$$

- É uma métrica de **distância generalizada**, ou seja, podemos alterar o parâmetro p para calcular a distância entre dois pontos de formas diferentes.

Distância de Minkowski: Casos particulares



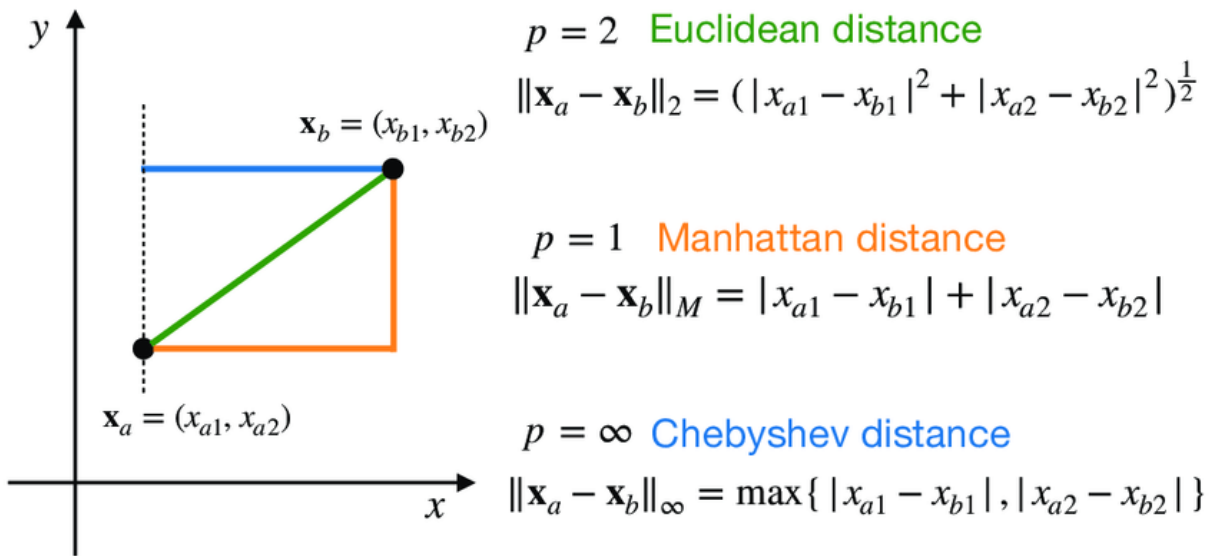
- Para $p = 1$, temos a ***distância de Manhattan***:

$$d(\mathbf{x}; \mathbf{y}) = \sum_{i=1}^K |x_i - y_i|.$$

- Para $p = 2$, temos a ***distância Euclidiana***:

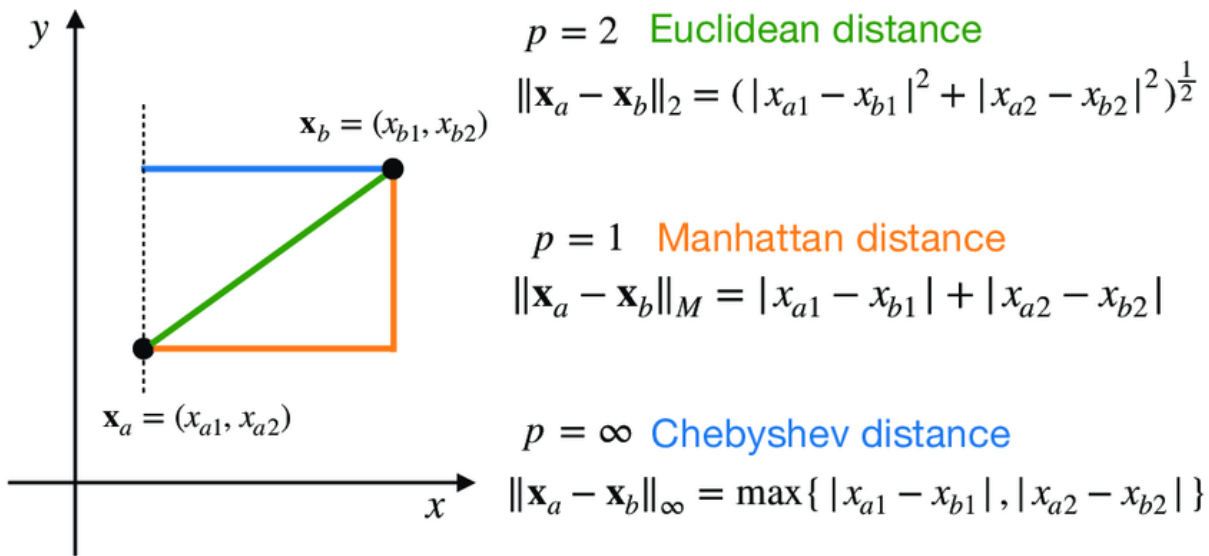
$$d(\mathbf{x}; \mathbf{y}) = \sqrt{\sum_{i=1}^K |x_i - y_i|^2}.$$

Distância Euclidiana



- Mede a **menor distância direta entre dois pontos** em um espaço, como se fosse uma **linha reta**.
- A distância Euclidiana é usada quando os **atributos têm uma relação linear**.

Distância Manhattan



- A distância de Manhattan mede a distância **movendo-se apenas ao longo de eixos horizontais e verticais**, como em uma grade de ruas de uma cidade (daí o nome Manhattan).
- A distância de Manhattan é mais adequada quando os **atributos não têm uma relação linear clara**.
- Usada quando o **movimento só pode ocorrer em direções perpendiculares**, como em redes urbanas ou circuitos.

k-NN para classificação

- Com relação ao problema da classificação, a saída da equação

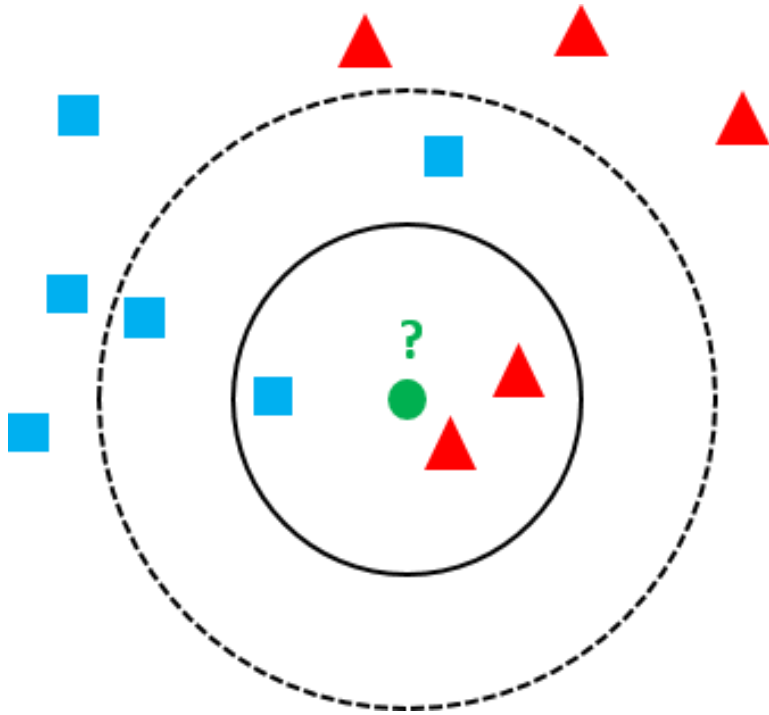
$$\hat{y}(\mathbf{x}') = \underset{q \in Q}{\operatorname{arg\,max}} \sum_{\mathbf{x}(i) \in N_k(\mathbf{x}')} \delta(q, y(i)),$$

Voto majoritário 

gerada pelo k-NN equivale a tomar o **voto majoritário** dos k vizinhos mais próximos de $\mathbf{x}'$, onde

- q é uma das classes do conjunto de classes Q ,
 - $N_k(\mathbf{x}')$ são os k vizinhos mais próximos de $\mathbf{x}'$, ou seja, os k exemplos de treinamento, $\mathbf{x}(i)$, mais próximos de $\mathbf{x}'$,
 - $y(i)$ são as classes correspondentes aos k vizinhos mais próximos de $\mathbf{x}'$,
 - $\delta(i, j)$ é o delta de Kronecker onde $\delta(i, j) = 1$ se $i == j$ e 0 caso contrário.
- Em resumo, um novo exemplo de entrada, $\mathbf{x}'$, é classificado como sendo pertencente à classe que contiver o maior número de vizinhos de $\mathbf{x}'$.

Exemplo de classificação com k-NN



- Na figura ao lado, o exemplo de teste (**ponto verde**) pode ser classificado como pertencente à classe dos **quadrados azuis** ou à classe dos **triângulos vermelhos** dependendo do valor de k .
- Se $k = 3$ (círculo com linha sólida), ele é atribuído à classe dos triângulos vermelhos pois existem 2 triângulos e apenas 1 quadrado dentro do círculo interno.
- Se $k = 5$ (círculo tracejado), ele é atribuído à classe dos quadrados azuis (3 quadrados vs. 2 triângulos dentro do círculo externo).

Exemplo de classificação com k-NN

- Uma desvantagem da classificação por **votação majoritária** ocorre quando a **distribuição das classes é desbalanceada**.
- Ou seja, **exemplos de uma classe mais frequente tendem a dominar a predição de um exemplo de entrada**, pois tendem a ser comuns entre os k vizinhos mais próximos devido ao seu maior número.
- Portanto, nestas circunstâncias, uma técnica bastante utilizada para se classificar os exemplos de entrada é **atribuir pesos diferentes à contribuição de cada vizinho à decisão final** de tal forma que **vizinhos mais próximos contribuam mais do que vizinhos mais distantes**.
- Uma forma usual é definir os **pesos** como sendo **inversamente proporcionais às distâncias dos vizinhos ao exemplo de entrada, x'** .

k-NN para regressão

- Seja $N_k(\mathbf{x}')$ o conjunto formado pelos k exemplos de treinamento, $\mathbf{x}(j) \in \mathbb{R}^{K \times 1}$, $j = 1, \dots, k$, mais próximos ao exemplo de entrada $\mathbf{x}'$.
- As saídas associadas (i.e., rótulos) a estes exemplos de treinamento são denotadas por $y_j(\mathbf{x} \in N_k(\mathbf{x}'))$, $j = 1, \dots, k$.
- Desta forma, quando utilizado para **regressão**, a saída do algoritmo k-NN para um novo exemplo de entrada, $\mathbf{x}'$, pode ser escrita de forma geral como

$$\hat{y}(\mathbf{x}') = \frac{\sum_{j=1}^k w_j y_j(\mathbf{x} \in N_k(\mathbf{x}'))}{\sum w_j},$$

onde w_j , $j = 1, \dots, k$ representa o peso associado ao j -ésimo vizinho de $\mathbf{x}'$.

- Os pesos associados aos vizinhos podem ser **uniformes** ou **inversamente proporcionais à distância**.

Exemplo

- Exemplo: knn.ipynb

